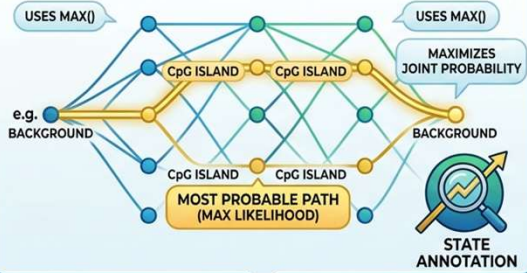


COMPARING HMM ALGORITHMS

DNA SEQUENCE (OBSERVED EMISSIONS)

**VITERBI ALGORITHM: PATH FINDING (DECODING)**

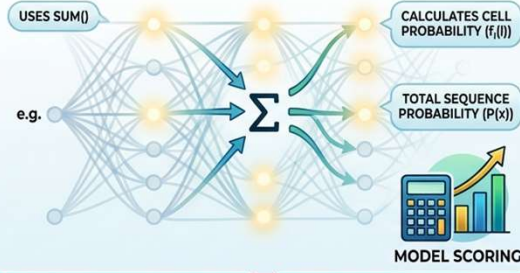
GOAL: FIND THE SINGLE MOST PROBABLE STATE PATH



BG-BG-CpG-CpG-BG
ANNOTATED PATH

FORWARD ALGORITHM: SCORE CALCULATION (EVALUATION)

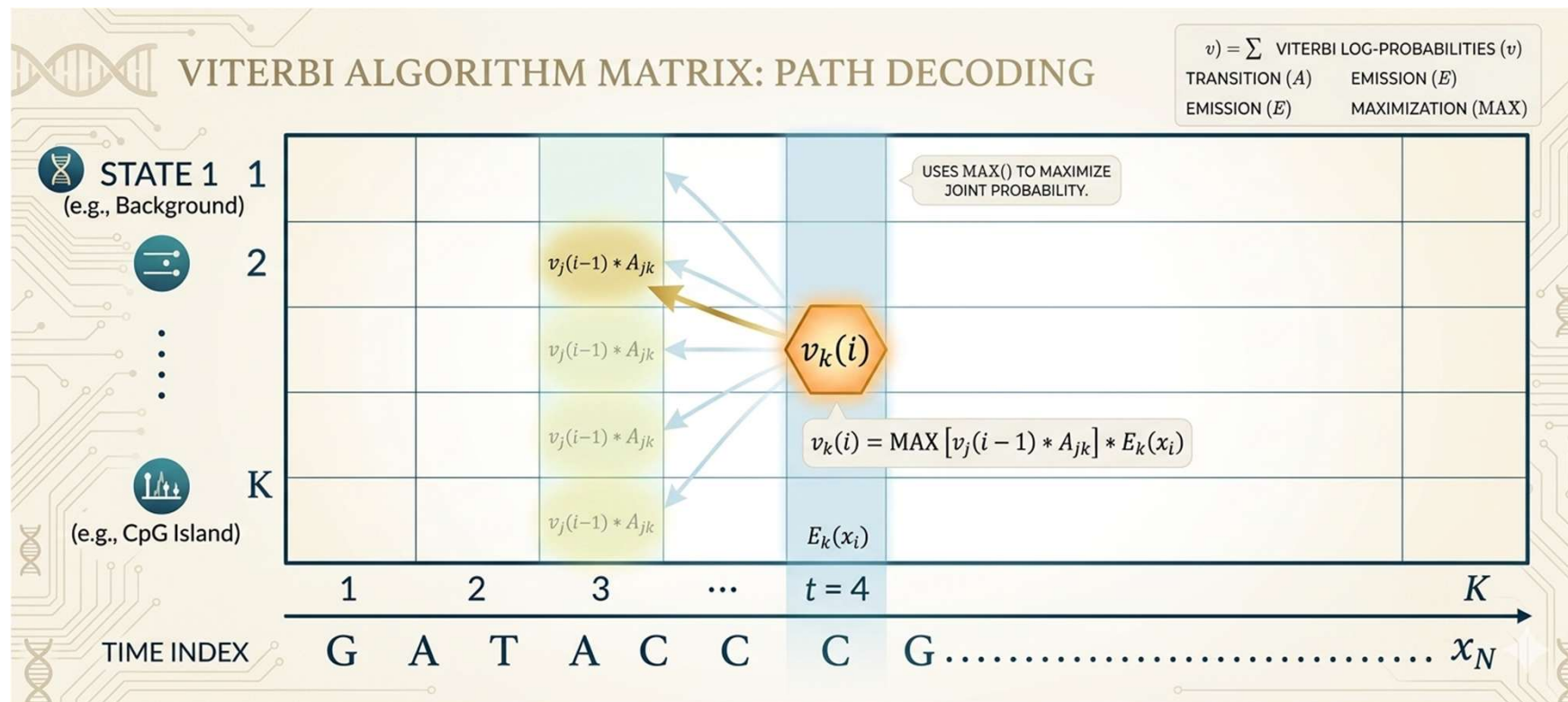
GOAL: SUM PROBABILITIES OF ALL POSSIBLE PATHS



0.00394
TOTAL SCORE ($P(x)$)

Lecture 11**HIDDEN MARKOV MODELS III**
Comparing HMM algorithms

1. Algoritmické nastavenia pre HMM



Rekurzia ($i = 1 :: N$): $v_k(i) = e_k(x_i) \max_j (a_{jk} v_j(i-1))$; $ptr_i(l) = \arg \max_j (a_{jk} v_j(i-1))$.

2. Viterbiho algoritmus pre genomicky scanner

Model: „Genomický skener“

Tento príklad rozšírime na **3 stavy**, čo je typický model pre bioinformatickú analýzu DNA sekvencií.

Model: „Genomický skener“

Máme 3 skryté stavy:

1. **B (Background)**: Pozadie genómu (bežný výskyt nukleotidov).
2. **C (CpG Island)**: Oblasť bohatá na C a G.
3. **S (Start Motif)**: Krátka regulačná oblasť (napr. TATA box), ktorá často predchádza gény.

Parametre modelu:

- **Štart:** $P(B) = 0.4, P(C) = 0.2, P(S) = 0.4$
- **Prechody (Matica A):**
 - $Z\ B: P(B \rightarrow B) = 0.7, P(B \rightarrow C) = 0.2, P(B \rightarrow S) = 0.1$
 - $Z\ C: P(C \rightarrow B) = 0.1, P(C \rightarrow C) = 0.8, P(C \rightarrow S) = 0.1$
 - $Z\ S: P(S \rightarrow B) = 0.4, P(S \rightarrow C) = 0.4, P(S \rightarrow S) = 0.2$
- **Emisné pravdepodobnosti (zjednodušené):**
 - Pre $B: P(G|B) = 0.2, P(A|B) = 0.3$
 - Pre $C: P(G|C) = 0.4, P(A|C) = 0.1$
 - Pre $S: P(G|S) = 0.25, P(A|S) = 0.4$

Pozorovaná sekvencia (x): {G, A, G}

2. Viterbiho algoritmus pre genomicky scanner

Model: „Genomický skener“

Tento príklad rozšírime na **3 stavy**, čo je typický model pre bioinformatickú analýzu DNA sekvencií.

Model: „Genomický skener“

Máme 3 skryté stavy:

1. **B (Background):** Pozadie genómu (bežný výskyt nukleotidov).
2. **C (CpG Island):** Oblasť bohatá na C a G.
3. **S (Start Motif):** Krátka regulačná oblasť (napr. TATA box), ktorá často predchádza gény.

Parametre modelu:

- **Štart:** $P(B) = 0.4, P(C) = 0.2, P(S) = 0.4$
- **Prechody (Matica A):**
 - Z B: $P(B \rightarrow B) = 0.7, P(B \rightarrow C) = 0.2, P(B \rightarrow S) = 0.1$
 - Z C: $P(C \rightarrow B) = 0.1, P(C \rightarrow C) = 0.8, P(C \rightarrow S) = 0.1$
 - Z S: $P(S \rightarrow B) = 0.4, P(S \rightarrow C) = 0.4, P(S \rightarrow S) = 0.2$
- **Emisné pravdepodobnosti (zjednodušené):**
 - Pre B: $P(G|B) = 0.2, P(A|B) = 0.3$
 - Pre C: $P(G|C) = 0.4, P(A|C) = 0.1$
 - Pre S: $P(G|S) = 0.25, P(A|S) = 0.4$

Pozorovaná sekvencia (x): {G, A, G}

Krok 1: Čas $t = 1$ (Pozorovanie: 'G')

Vypočítame počiatkové pravdepodobnosti pre každý stav.

- $v_B(1) = P(B) \cdot P(G|B) = 0.4 \cdot 0.2 = \mathbf{0.08}$
- $v_C(1) = P(C) \cdot P(G|C) = 0.2 \cdot 0.4 = \mathbf{0.08}$
- $v_S(1) = P(S) \cdot P(G|S) = 0.4 \cdot 0.25 = \mathbf{0.10}$

Krok 2: Čas $t = 2$ (Pozorovanie: 'A')

Teraz pre každý stav ($k \in \{B, C, S\}$) hľadáme najlepšieho predchodcu.

Vzorec: $v_k(2) = P(A|k) \cdot \max_j [v_j(1) \cdot A_{jk}]$

- **Pre stav B:**
 - Max: $\max(0.08 \cdot 0.7, 0.08 \cdot 0.1, 0.10 \cdot 0.4) = \max(0.056, 0.008, 0.04) = 0.056$ (z B)
 - $v_B(2) = P(A|B) \cdot 0.056 = 0.3 \cdot 0.056 = \mathbf{0.0168}$
- **Pre stav C:**
 - Max: $\max(0.08 \cdot 0.2, 0.08 \cdot 0.8, 0.10 \cdot 0.4) = \max(0.016, 0.064, 0.04) = 0.064$ (z C)
 - $v_C(2) = P(A|C) \cdot 0.064 = 0.1 \cdot 0.064 = \mathbf{0.0064}$
- **Pre stav S:**
 - Max: $\max(0.08 \cdot 0.1, 0.08 \cdot 0.1, 0.10 \cdot 0.2) = \max(0.008, 0.008, 0.02) = 0.02$ (z S)
 - $v_S(2) = P(A|S) \cdot 0.02 = 0.4 \cdot 0.02 = \mathbf{0.0080}$

2. Viterbiho algoritmus pre genomicky scanner

Krok 3: Čas $t = 3$ (Pozorovanie: 'G')

Hľadáme najlepšieho predchodcu z času $t = 2$.

Vzorec: $v_k(3) = P(G | k) \cdot \max_j [v_j(2) \cdot A_{jk}]$

- **Pre stav B:**

- Max:
 $\max(v_B(2) \cdot 0.7, v_C(2) \cdot 0.1, v_S(2) \cdot 0.4) = \max(0.01176, 0.00064, 0.0032) = 0.01176$
(z B)
- $v_B(3) = P(G | B) \cdot 0.01176 = 0.2 \cdot 0.01176 = \mathbf{0.002352}$

- **Pre stav C:**

- Max:
 $\max(v_B(2) \cdot 0.2, v_C(2) \cdot 0.8, v_S(2) \cdot 0.4) = \max(0.00336, 0.00512, 0.0032) = 0.00512$
(z C)
- $v_C(3) = P(G | C) \cdot 0.00512 = 0.4 \cdot 0.00512 = \mathbf{0.002048}$

- **Pre stav S:**

- Max:
 $\max(v_B(2) \cdot 0.1, v_C(2) \cdot 0.1, v_S(2) \cdot 0.2) = \max(0.00168, 0.00064, 0.0016) = 0.00168$
(z B)
- $v_S(3) = P(G | S) \cdot 0.00168 = 0.25 \cdot 0.00168 = \mathbf{0.00042}$

Krok 4: Ukončenie a Spätné sledovanie (Backtracking)

1. Porovnanie koncov:

- $v_B(3) = 0.002352$
- $v_C(3) = 0.002048$
- $v_S(3) = 0.00042$
- Najvyšší je stav **B** (Background).

2. Backtracking:

- Stav v $t = 3$ je **B**.
- Pozrieme sa, odkiaľ prišlo $v_B(3)$: Prišlo zo stavu **B** v čase $t = 2$.
- Pozrieme sa, odkiaľ prišlo $v_B(2)$: Prišlo zo stavu **B** v čase $t = 1$.

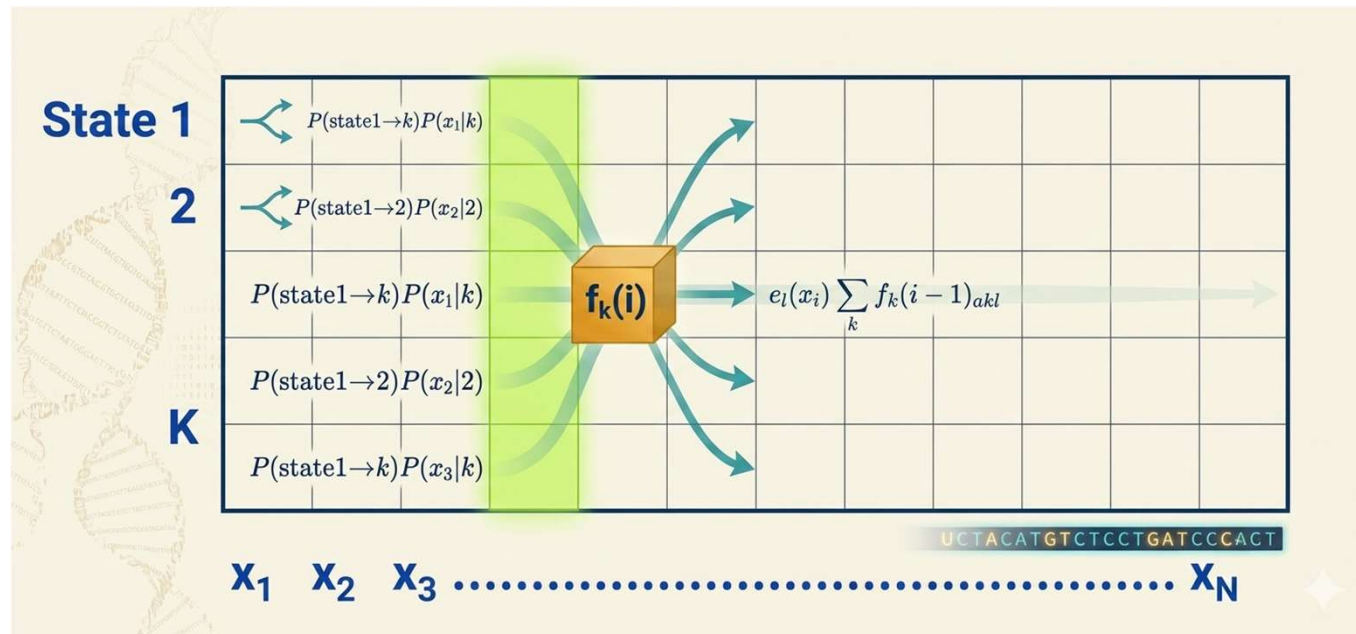
Výsledná najpravdepodobnejšia cesta: $B \rightarrow B \rightarrow B$

Prečo algoritmus vybral "B"?

Aj keď v sekvencii boli G-čka (ktoré má rada C) a A-čko, model je nastavený tak, že **zotrvanie v stave Background (0.7)** je veľmi pravdepodobné. Prepnutie do CpG ostrova (C) alebo regulačnej oblasti (S) "stojí" príliš veľa pravdepodobnosti vzhľadom na krátku sekvenciu. Keby sekvencia pokračovala ďalej (napr. ďalšie 3x 'G'), algoritmus by s najväčšou pravdepodobnosťou "prepol" do stavu C.

Tento ručný výpočet vám ukázal, že **Viterbi nie je len o maximách emisií, ale o rovnováhe medzi emisiami a pravdepodobnosťou prechodu.**

1. Algoritmické nastavenia pre HMM



Z obrázku možno vidieť, že **Forward algoritmus** je veľmi podobný **Viterbiho algoritmu**, okrem toho, že **vo Forward algoritme sa namiesto maximalizácie používa sčítanie**. Môžeme teda znovu použiť výpočty z predchádzajúceho problému vrátane penalizácie emisií, penalizácie prechodov a súčtov predchádzajúcich stavov.

Požadovaný výpočtový čas je $O(K^2N)$ a požadovaný priestor je $O(KN)$.

Nevýhodou tohto algoritmu je, že v praxi je sčítanie logaritmov zložité; preto sa namiesto toho používajú aproximácie a škálovanie pravdepodobností.

1. Algoritmické nastavenia pre HMM

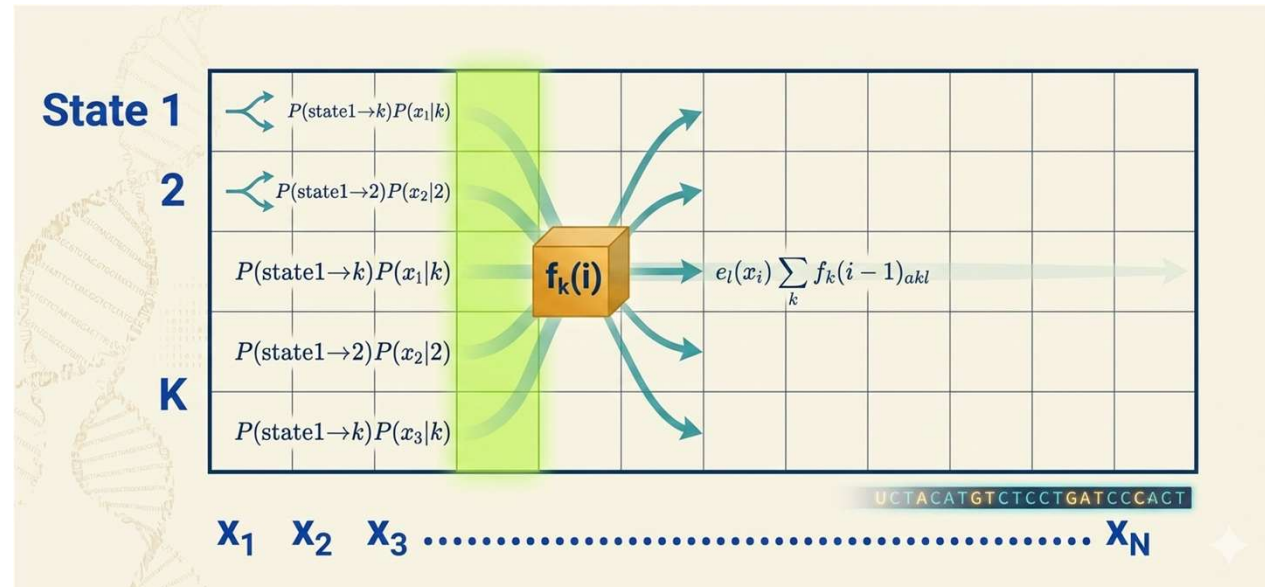
Forward algoritmus

Najprv definujeme doprednú (forward) pravdepodobnosť $f_i(i)$:

$$\begin{aligned} f_i(i) &= P(x_1 \dots x_i \mid \pi = l) \\ &= \sum_{\pi_1 \dots \pi_{i-1}} P(x_1 \dots x_{i-1} \mid (\pi_1, \dots, \pi_{i-2}, \pi_{i-1}); \pi_i = l) e_l(x_i) \\ &= \sum_k \sum_{\pi_1 \dots \pi_{i-2}} P(x_1 \dots x_{i-1} \mid (\pi_1, \dots, \pi_{i-2}); \pi_{i-1} = k) a_{kl} e_l(x_i) \\ &= \sum_k f_k(i-1) a_{kl} e_l(x_i) \\ &= e_l(x_i) \sum_k f_k(i-1) a_{kl} \end{aligned}$$

Celý algoritmus je tento:

- **Inicializácia** ($i = 0$): $f_0(0) = 1, f_k(0) = 0$ pre $k > 0$.
- **Iterácia** ($i = 1 \dots N$): $f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$.
- **Ukončenie**: $P(x, \pi^*) = \sum_k f_k(N)$



2. Forward algoritmus pre genomicky scanner

Forward algoritmus

Forward algoritmus je **súčet pravdepodobností všetkých možných ciest**. Zatiaľ čo Viterbi hľadá tú jednu „najlepšiu“ cestu (maximalizuje), Forward algoritmus sa pýta: „Aká je celková pravdepodobnosť, že tento model vygeneroval túto sekvenciu, keď zoberiem do úvahy všetky kombinácie stavov?“

V matematickom zápise je jediný rozdiel: **namiesto operácie max používame sum (súčet)**.

Použijeme úplne rovnaké parametre (stavy B, C, S a sekvenciu $\{G, A, G\}$) ako v predchádzajúcom príklade, aby ste videli ten rozdiel.

Parametre modelu (zhrnutie):

- **Stavy:** B, C, S
- **Počiatkové pravdepodobnosti (π):** $B = 0.4, C = 0.2, S = 0.4$
- **Matica prechodov (A):**
 - $B \rightarrow \{B : 0.7, C : 0.2, S : 0.1\}$
 - $C \rightarrow \{B : 0.1, C : 0.8, S : 0.1\}$
 - $S \rightarrow \{B : 0.4, C : 0.4, S : 0.2\}$
- **Emisné pravdepodobnosti (E):**
 - $B : \{G : 0.2, A : 0.3\}$
 - $C : \{G : 0.4, A : 0.1\}$
 - $S : \{G : 0.25, A : 0.4\}$

Krok 1: Inicializácia ($t = 1$, pozorovaný znak $x_1 = G$)

Vypočítame pravdepodobnosť, že sme v prvom kroku v konkrétnom stave a vyemitovali sme G .

Vzorec: $f_k(1) = \pi_k \cdot P(G|k)$

- $f_B(1) = P(B) \cdot P(G|B) = 0.4 \cdot 0.2 = \mathbf{0.08}$
- $f_C(1) = P(C) \cdot P(G|C) = 0.2 \cdot 0.4 = \mathbf{0.08}$
- $f_S(1) = P(S) \cdot P(G|S) = 0.4 \cdot 0.25 = \mathbf{0.1}$

Krok 2: Rekurzia ($t = 2$, pozorovaný znak $x_2 = A$)

Teraz vypočítame pravdepodobnosť pre každý stav v čase $t = 2$.

Vzorec: $f_l(t) = P(A|l) \cdot \sum_{k \in \{B, C, S\}} (f_k(t-1) \cdot A_{kl})$

Výpočet pre $f_B(2)$ (Stav B):

$$\begin{aligned} f_B(2) &= P(A|B) \cdot [f_B(1) \cdot A_{BB} + f_C(1) \cdot A_{CB} + f_S(1) \cdot A_{SB}] \\ f_B(2) &= 0.3 \cdot [0.08 \cdot 0.7 + 0.08 \cdot 0.1 + 0.1 \cdot 0.4] \\ f_B(2) &= 0.3 \cdot [0.056 + 0.008 + 0.04] = 0.3 \cdot 0.104 = \mathbf{0.0312} \end{aligned}$$

Výpočet pre $f_C(2)$ (Stav C):

$$\begin{aligned} f_C(2) &= P(A|C) \cdot [f_B(1) \cdot A_{BC} + f_C(1) \cdot A_{CC} + f_S(1) \cdot A_{SC}] \\ f_C(2) &= 0.1 \cdot [0.08 \cdot 0.2 + 0.08 \cdot 0.8 + 0.1 \cdot 0.4] \\ f_C(2) &= 0.1 \cdot [0.016 + 0.064 + 0.04] = 0.1 \cdot 0.12 = \mathbf{0.012} \end{aligned}$$

Výpočet pre $f_S(2)$ (Stav S):

$$\begin{aligned} f_S(2) &= P(A|S) \cdot [f_B(1) \cdot A_{BS} + f_C(1) \cdot A_{CS} + f_S(1) \cdot A_{SS}] \\ f_S(2) &= 0.4 \cdot [0.08 \cdot 0.1 + 0.08 \cdot 0.1 + 0.1 \cdot 0.2] \\ f_S(2) &= 0.4 \cdot [0.008 + 0.008 + 0.02] = 0.4 \cdot 0.036 = \mathbf{0.0144} \end{aligned}$$

2. Forward algoritmus pre genomicky scanner

Krok 3: Rekurgia ($t = 3$, pozorovaný znak $x_3 = G$)

Opäť použijeme rovnaký princíp rekurgie pre tretí znak.

Výpočet pre $f_B(3)$ (Stav B):

$$\begin{aligned} f_B(3) &= P(G|B) \cdot [f_B(2) \cdot 0.7 + f_C(2) \cdot 0.1 + f_S(2) \cdot 0.4] \\ f_B(3) &= 0.2 \cdot [0.0312 \cdot 0.7 + 0.012 \cdot 0.1 + 0.0144 \cdot 0.4] \\ f_B(3) &= 0.2 \cdot [0.02184 + 0.0012 + 0.00576] = 0.2 \cdot 0.0288 = \mathbf{0.00576} \end{aligned}$$

Výpočet pre $f_C(3)$ (Stav C):

$$\begin{aligned} f_C(3) &= P(G|C) \cdot [f_B(2) \cdot 0.2 + f_C(2) \cdot 0.8 + f_S(2) \cdot 0.4] \\ f_C(3) &= 0.4 \cdot [0.0312 \cdot 0.2 + 0.012 \cdot 0.8 + 0.0144 \cdot 0.4] \\ f_C(3) &= 0.4 \cdot [0.00624 + 0.0096 + 0.00576] = 0.4 \cdot 0.0216 = \mathbf{0.00864} \end{aligned}$$

Výpočet pre $f_S(3)$ (Stav S):

$$\begin{aligned} f_S(3) &= P(G|S) \cdot [f_B(2) \cdot 0.1 + f_C(2) \cdot 0.1 + f_S(2) \cdot 0.2] \\ f_S(3) &= 0.25 \cdot [0.0312 \cdot 0.1 + 0.012 \cdot 0.1 + 0.0144 \cdot 0.2] \\ f_S(3) &= 0.25 \cdot [0.00312 + 0.0012 + 0.00288] = 0.25 \cdot 0.0072 = \mathbf{0.0018} \end{aligned}$$

Záver: Celková pravdepodobnosť

Aby sme získali celkovú pravdepodobnosť, že model vygeneroval sekvenciu $\{G, A, G\}$, sčítame výsledky z posledného kroku:

$$\begin{aligned} P(x) &= f_B(3) + f_C(3) + f_S(3) \\ P(x) &= 0.00576 + 0.00864 + 0.0018 = \mathbf{0.0162} \end{aligned}$$

Výsledok: Celková pravdepodobnosť sekvencie $\{G, A, G\}$ pri tomto modeli je **0.0162**.

Hlavný rozdiel v filozofii

- Forward algoritmus (Agregátor):** Pýta sa: „Aká je **celková pravdepodobnosť**, že tieto dáta vznikli v mojom modeli?“ Sčítava pravdepodobnosti všetkých možných ciest (scenárov).
- Viterbi algoritmus (Optimalizátor):** Pýta sa: „Aká je **najpravdepodobnejšia cesta** (postupnosť stavov), ktorá viedla k týmto dátam?“ Zahadzuje všetky ostatné cesty a ponecháva si len tú „vítaznú“.

Porovnávací tabuľka

Vlastnosť	Forward Algoritmus	Viterbi Algoritmus
Matematická operácia	Suma ($\sum$)	Maximum (max)
Výstup	Celková pravdepodobnosť $P(x)$	Pravdepodobnosť najlepšej cesty
Účel	Ohodnotenie modelu (ako dobre fitujú dáta)	Identifikácia skrytej sekvencie (dešifrovanie)
Výsledok pre G,A,G	0.0162	0.002352

Kódovanie pamäte v HMM - Detekcia CpG ostrovčekov

Môžeme do nášho modelu zahrnúť pamäť?

Odpoveď na túto otázku znie – Áno, môžeme!

Ale ako? **Pripomeňme si, že Markovove modely sú bezpamäťové.** Inými slovami, všetka pamäť modelu je obsiahnutá v stavoch. **Takže, aby sme mohli uložiť dodatočné informácie, musíme zvýšiť počet stavov.**

Teraz sa pozrime späť na biologický príklad. V našom modeli emisie stavov záviseli iba od aktuálneho stavu. A aktuálny stav kodoval iba jeden nukleotid. Čo však ak chceme, aby náš model počítal frekvencie dinukleotidov (pre CpG ostrovčeky¹), alebo trinukleotidové frekvencie (pre kodóny), alebo dikodónové frekvencie zahrňajúce šesť nukleotidov?

CpG ostrovčeky sú definované ako oblasti v genóme, ktoré sú obohatené o páry nukleotidov C a G na tom istom vlákne.

- Typicky, keď je tento dinukleotid prítomný v genóme, metyluje sa a keď dôjde k deaminácii cytozínu, ako sa to deje pri určitej základnej frekvencii, stane sa tymínom, ďalším prirodzeným nukleotidom, a preto ho bunka nemôže tak ľahko rozpoznať ako mutáciu, čo spôsobuje mutáciu C na T.
- Táto zvýšená frekvencia mutácií na CpG ostrovčekoch v priebehu evolučného času vyčerpáva CpG ostrovčeky a robí ich relatívne vzácnymi.
- Keďže metylácia môže nastať na ktoromkoľvek vlákne, CpG zvyčajne mutujú na TpG alebo CpA.
- **Ak sú však umiestnené v aktívnom promótore, metylácia je potlačená a CpG dinukleotidy sú schopné pretrvávať.**
- **Podobne sú CpG v oblastiach dôležitých pre funkciu bunky konzervované vďaka evolučnému tlaku.** V dôsledku toho môže detekcia CpG ostrovčekov zvýrazniť oblasti promótora, iné transkripčne aktívne oblasti alebo miesta purifikačnej selekcie v genóme.

Vedeli ste?

CpG je skratka pre [C]ytozín - [p]fosfátový reťazec - [G]uanín. Písmeno „p“ znamená, že hovoríme o tom istom vlákne dvojitej špirály, a nie o bázo-
vom páre GC, ktorý sa nachádza naprieč špirálou.

4. Špeciálne otázky

Vzhľadom na ich biologický význam sú CpG ostrovčeky hlavnými kandidátmi na modelovanie.

- Spočiatku sa možno pokúsiť identifikovať tieto ostrovčeky skenovaním genómu a hľadať v nich fixné intervaly bohaté na GC. Účinnosť tohto prístupu je ohrozená výberom vhodnej veľkosti okna; zatiaľ čo príliš malé okno nemusí zachytiť celý konkrétny CpG ostrovček, príliš veľké okno by viedlo k vynechaniu mnohých menších, ale skutočných CpG ostrovčekov.
- Skúmanie genómu na základe kodónov tiež vedie k ťažkostiam, pretože páry CpG nemusia nevyhnutne kódovať aminokyseliny, a preto nemusia ležať v jednom kodóne.
- Namiesto toho sú HMM oveľa vhodnejšie na modelovanie tohto scenára, pretože, ako čoskoro uvidíme v časti o unsupervised učení, HMM môžu prispôbiť svoje základné parametre tak, aby maximalizovali svoju pravdepodobnosť.

Nie všetky HMM sú však na túto konkrétnu úlohu vhodné.

Model HMM, ktorý zohľadňuje iba frekvencie jednotlivých nukleotidov C a G, nedokáže zachytiť povahu CpG ostrovčekov.

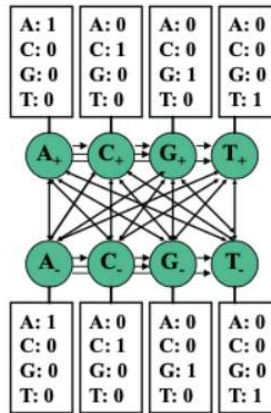
Uvažujme jeden takýto HMM s dvoma nasledujúcimi skrytými stavmi:

- Stav „+“ predstavuje ostrovy CpG
- Stav „-“: predstavuje neostrovné oblasti

Každý z týchto dvoch stavov potom s určitou pravdepodobnosťou emituje bázy A, C, G a T. Hoci ostrovčeky CpG v tomto modeli môžu byť obohatené o bázy C a G zvýšením ich príslušných emisných pravdepodobností, tento model nedokáže zachytiť skutočnosť, že C a G sa vyskytujú prevažne v pároch.

4. Špeciálne otázky

- **Vzhľadom na Markovovu vlastnosť, ktorá riadi HMM, jediné informácie dostupné v každom časovom kroku musia byť obsiahnuté v aktuálnom stave.**
- **Preto na kódovanie pamäte v rámci Markovovho reťazca musíme rozšíriť stavový priestor.** Na to je možné jednotlivé stavy „+“ a „-“ nahradiť 4 stavmi „+“ a 4 stavmi „-“: A+, C+, G+, T+, A-, C-, G-, T-



Konkrétne existujú 2 spôsoby modelovania a táto voľba bude mať za následok rôzne pravdepodobnosti emisií:

- Jeden model naznačuje, že stav A₊ napríklad znamená, že sa momentálne nachádzame v CpG ostrove a predchádzajúci znak bol A. Pravdepodobnosti emisií tu budú niest väčšinu informácií a prechody budú dosť degenerované.
- Iný model naznačuje, že stav A₊ napríklad znamená, že sa momentálne nachádzame v CpG ostrove a aktuálny znak je A. Pravdepodobnosť emisie tu bude 1 pre A a 0 pre všetky ostatné písmená a pravdepodobnosti prechodu budú niest väčšinu informácií v modeli a emisie budú dosť degenerované. Odteraz budeme predpokladať tento model.

4. Špeciálne otázky

Vedeli ste?

- **Počet prechodov je druhá mocnina počtu stavov.** To dáva hrubú predstavu o tom, ako sa zvyšuje „pamäť“ (a teda aj stavy) HMM.
- **Pamäť tohto systému je odvodená zo skutočnosti, že každý stav môže emitovať iba jeden znak, a preto si svoj emitovaný znak „pamätá“.**
- Okrem toho je dinukleotidová povaha CpG ostrovčekov začlenená do prechodových matic. Najmä frekvencia prechodov zo stavov C+ do G+ je výrazne vyššia ako zo stavov C- do G-, čo dokazuje, že tieto páry sa v rámci ostrovčekov vyskytujú častejšie.

Otázka: Keďže každý stav vydáva iba jeden znak, môžeme povedať, že sa to redukuje na Markovov reťazec namiesto HMM?

Nie. Aj keď emisie označujú písmeno skrytého stavu, neindikujú, či je stav CpG ostrovom alebo nie: stav A- aj A+ emitujú iba pozorovateľný stav A.

Otázka: Ako môžeme začleniť naše znalosti o systéme pri tréňovaní HMM modelov, napr. niektoré pravdepodobnosti emisií 0 v prípade detekcie CpG ostrova?

Mohli by sme buď vnútiť modelu naše znalosti nastavením niektorých parametrov a ponechať iné, aby sa menili, alebo by sme mohli nechať HMM voľne pracovať s modelom a nechať ho objaviť tieto vzťahy. V skutočnosti existujú dokonca metódy, ktoré model zjednodušujú tým, že vynútiť podmnožinu parametrov na 0, ale umožnia HMM vybrať si, ktorú podmnožinu.

Vzhľadom na vyššie uvedený rámec môžeme použiť posteriórne dekódovanie na analýzu každej bázy v genóme a určenie, či je s najväčšou pravdepodobnosťou súčasťou CpG ostrova alebo nie. Ale po zostavení rozšíreného HMM modelu, ako môžeme overiť, či je v skutočnosti lepší ako model s jedným nukleotidom? Predtým sme demonštrovali, že algoritmus dopredu alebo dozadu možno použiť na výpočet $P(x)$ pre daný model. **Ak je pravdepodobnosť nášho súboru údajov vyššia pri druhom modeli ako pri prvom modeli, s najväčšou pravdepodobnosťou efektívnejšie zachytáva základné správanie.**

4. Špeciálne otázky

Existuje však jedno riziko spojené s komplikáciou modelu, a to preusporiadanie.

- Zvýšenie počtu parametrov pre HMM zvyšuje pravdepodobnosť preusporiadania dát zo strany HMM a znižuje presnosť zachytenia základného správania.
- Bežným riešením v strojovom učení je použitie **regularizácie**, čo v podstate znamená použitie menšieho počtu parametrov. V tomto prípade je možné znížiť počet parametrov, ktoré sa majú naučiť, obmedzením všetkých pravdepodobností prechodu +/- na rovnakú hodnotu a všetkých pravdepodobností prechodu -/+ na rovnakú hodnotu, pretože prechody tam a späť z + a - stavov sú to, čo nás zaujíma pri modelovaní, a skutočné bázy, kde k prechodu došlo, nie sú pre náš model až také dôležité. Preto sa pre tento obmedzený model musíme naučiť menej parametrov, čo vedie k jednoduchšiemu modelu a môže pomôcť vyhnúť sa preusporiadaniu.

Otázka: Existujú aj iné spôsoby kódovania pamäte pre detekciu CpG ostrovčekov?

Odpoveď: Ďalšie nápady, s ktorými by sa dalo experimentovať, zahŕňajú

- vyhľadávajúte dinukleotidy a nájdite spôsob, ako sa vysporiadať s prekrývaním.
- pridajte špeciálny stav, ktorý prechádza z C do G.

Posterior decoding

- Táto prednáška bude diskutovať o posteriornom dekodovaní (posterior decoding), **algoritme, ktorý opäť odvodí sekvenciu skrytých stavov π , ktorá maximalizuje odlišnú metriku.**
- Konkrétne, nájde najpravdepodobnejší stav na každej pozícii naprieč všetkými možnými cestami, a to pomocou dopredného aj spätného algoritmu.
- Následne ukážeme, ako zakódovať „pamäť“ do Markovovho reťazca pridaním ďalších stavov, aby sme prehľadali genóm pre stavy uchovávajúce dinukleotidy.
- Potom budeme diskutovať o tom, ako použiť odhad parametrov metódou maximálnej vierohodnosti (Maximum Likelihood) pre supervizované učenie s označenou množinou dát.
- Stručne uvidíme, ako použiť Viterbiho učenie pre nesupervizovaný odhad parametrov neoznačenej množiny dát.
- Nakoniec sa naučíme, ako použiť algoritmus očakávania a maximalizácie (Expectation Maximization – EM) pre nesupervizovaný odhad parametrov neoznačenej množiny dát, kde špecifický algoritmus pre HMM je známy ako Baum-Welchov algoritmus.

Posteriórne dekódovanie (Posterior Decoding)

Motivácia

Hoci Viterbiho algoritmus dekódovania poskytuje jeden spôsob odhadu skrytých stavov podkladajúcich sekvenciu pozorovaných znakov, ďalší platný spôsob inferencie (odvodzovania) poskytuje **posteriórne dekódovanie**.

Posteriórne dekódovanie poskytuje najpravdepodobnejší stav v akomkoľvek časovom bode.

Aby sme získali určitú intuíciu pre posteriórne dekódovanie, pozrime sa, ako sa aplikuje na **situáciu, v ktorej nepoctivé kasíno strieda férovú a cinknutú kocku**. Predpokladajme, že vstúpime do kasína s vedomím, že nefér (cinknutá) kocka sa používa 60 percent času. S touto znalosťou a bez akýchkoľvek hodov kockou je náš najlepší odhad pre aktuálnu kocku zjavne tá cinknutá. Po jednom hode je pravdepodobnosť, že bola použitá cinknutá kocka, daná vzorcom:

$$P(\text{kocka} = \text{cinknutá} \mid \text{hod} = k) = \frac{P(\text{kocka} = \text{cinknutá}) \cdot P(\text{hod} = k \mid \text{kocka} = \text{cinknutá})}{P(\text{hod} = k)}$$

Ak by sme namiesto toho pozorovali sekvenciu N hodov kockou, ako by sme vykonali podobný druh inferencie? Tým, že **dovolíme informáciám prúdiť medzi N hodmi a ovplyvňovať pravdepodobnosť každého stavu**, je posteriórne dekódovanie prirodzeným rozšírením vyššie uvedenej inferencie na sekvenciu ľubovoľnej dĺžky.

5. Posterior Decoding

Formálnejšie povedané, namiesto identifikácie jednej cesty maximálnej vierohodnosti, posteriórne dekodovanie uvažuje o pravdepodobnosti, že sa akákoľvek cesta nachádza v stave k v čase t pri daných všetkých pozorovaných znakoch, t. j. $P(\pi_t = k \mid x_1; \dots; x_n)$.

Stav, ktorý maximalizuje túto pravdepodobnosť pre daný čas, sa potom považuje za najpravdepodobnejší stav v tomto bode.

Je dôležité poznamenať, že okrem informácií prúdiacich dopredu na určenie najpravdepodobnejšieho stavu v bode, môžu informácie prúdiť aj spätne od konca sekvencie k tomuto stavu, aby sa zvýšila alebo znížila vierohodnosť každého stavu v danom bode. Je to čiastočne prirodzený dôsledok reverzibility Bayesovho pravidla: naše pravdepodobnosti sa pri pozorovaní ďalších údajov menia z apriórnych pravdepodobností na posteriórne pravdepodobnosti. Aby sme to objasnili, predstavte si opäť príklad s kasínom.

Ako bolo uvedené skôr, bez pozorovania akýchkoľvek hodov je stav0 s najväčšou pravdepodobnosťou nefér: toto je naša apriórna pravdepodobnosť. Ak je prvý hod 6, naše presvedčenie, že stav1 je nefér, sa posilní (ak je hádzanie šestiek pravdepodobnejšie pri nefér kocke). Ak sa opäť hodí 6, informácia prúdi spätne od druhého hodu kockou a posilňuje naše presvedčenie o stave1 o nefér kocke ešte viac. Čím viac hodov máme, tým viac informácií prúdi spätne a posilňuje alebo kontrastuje naše presvedčenia o stave, čím ilustruje spôsob, akým informácie prúdia spätne a dopredu, aby ovplyvnili naše presvedčenie o stavoch pri posteriórnom dekodovaní.

5. Posterior Decoding

Pomocou niekoľkých elementárnych úprav môžeme túto pravdepodobnosť preusporiadať do nasledujúceho tvaru pomocou Bayesovho pravidla:

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k \mid x_1; \dots; x_n) = \operatorname{argmax}_k \frac{P(\pi_t = k; x_1; \dots; x_n)}{P(x_1; \dots; x_n)}$$

Pretože $P(x)$ je konštanta, môžeme ju pri maximalizácii funkcie zanedbať. Preto,

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k; x_1; \dots; x_t) \cdot P(x_{t+1}; \dots; x_n \mid \pi_t = k; x_1; \dots; x_t)$$

Pomocou Markovovej vlastnosti môžeme tento výraz jednoducho zapísať nasledovne:

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k; x_1; \dots; x_t) \cdot P(x_{t+1}; \dots; x_n \mid \pi_t = k) = \operatorname{argmax}_k f_k(t) \cdot b_k(t)$$

Tu sme definovali

$$f_k(t) = P(\pi_t = k; x_1; \dots; x_t) \text{ a } b_k(t) = P(x_{t+1}; \dots; x_n \mid \pi_t = k).$$

Ako čoskoro uvidíme, tieto parametre sa vypočítavajú pomocou dopredného algoritmu a spätného algoritmu. **Na vyriešenie problému posteriórneho dekódovania stačí vyriešiť každý z týchto podproblémov.**

Dopredný algoritmus bol ilustrovaný v predchádzajúcej časti a spätný algoritmus bude vysvetlený v nasledujúcej časti.

5. Posterior Decoding

Odvozenie prechodu od vzťahu (1) k (2) je založené na dvoch hlavných matematických krokoch:

- zjednodušenie pomocou vlastností operátora **argmax** a
- rozklade spoločnej pravdepodobnosti pomocou pravidla reťazenia (**chain rule**).

Podme si to rozobrať krok za krokom.

1. Zjednodušenie pomocou operátora argmax

Východiskový vzťah (1) je:

$$\pi_i^* = \operatorname{argmax}_k \frac{P(\pi_i = k; x_1, \dots, x_n)}{P(x_1, \dots, x_n)}$$

Tu si musíme uvedomiť, čo hľadáme. Operátor **argmax** hľadá taký index (v našom prípade stav k), pre ktorý je hodnota výrazu **maximálna**.

- **Pozorovanie:** Menovateľ $P(x_1, \dots, x_n)$ predstavuje celkovú pravdepodobnosť pozorovanej sekvencie (všetkých dát). Táto hodnota **nezávisí od stavu** k . Je to konštanta pre všetky stavy, ktoré skúmame.
- **Dôsledok:** Ak chceme nájsť k , ktoré maximalizuje zlomok $f(k)/C$, je to to isté, ako keby sme hľadali k , ktoré maximalizuje iba čitateľ $f(k)$. Konštanta C nám poradie maximalizácie nezmení.

Preto môžeme menovateľ „zahodiť“ (zanedbať):

$$\pi_i^* = \operatorname{argmax}_k P(\pi_i = k; x_1, \dots, x_n)$$

5. Posterior Decoding

2. Rozklad pomocou pravidla reťazenia (Chain Rule)

Teraz máme spoločnú pravdepodobnosť $P(\pi_t = k; x_1, \dots, x_n)$. Chceme ju „rozstrihnúť“ v čase t na dve časti: **minulosť (a prítomnosť)** a **budúcnosť**.

Podľa definície podmienenej pravdepodobnosti platí:

$$P(A, B) = P(A) \cdot P(B | A)$$

Ak si definujeme:

- $A = (\pi_t = k; x_1, \dots, x_t)$ (stav v čase t a pozorované dáta do času t)
- $B = (x_{t+1}, \dots, x_n)$ (budúce pozorované dáta)

Potom spoločnú pravdepodobnosť celej sekvencie môžeme zapísať ako:

$$P(\underbrace{\pi_t = k; x_1, \dots, x_t}_A, \underbrace{x_{t+1}, \dots, x_n}_B) = P(\underbrace{\pi_t = k; x_1, \dots, x_t}_A) \cdot P(\underbrace{x_{t+1}, \dots, x_n}_B | \underbrace{\pi_t = k; x_1, \dots, x_t}_A)$$

Toto je presne to, čo potrebujeme na získanie vzťahu (2)

5. Posterior Decoding

3. Zostavenie výsledného vzťahu (2)

Keď spojíme tieto dva kroky, dostaneme:

$$\pi_t^* = \operatorname{argmax}_k [P(\pi_t = k; x_1, \dots, x_t) \cdot P(x_{t+1}, \dots, x_n | \pi_t = k; x_1, \dots, x_t)]$$

Toto je presne forma vzťahu (2).

Prečo je tento krok dôležitý?

Tento rozklad je základným „trikom“ algoritmu. Všimnite si dve časti:

1. $P(\pi_t = k; x_1, \dots, x_t)$: Toto je presne to, čo počíta **Forward algoritmus** (skóre dopredu).
2. $P(x_{t+1}, \dots, x_n | \pi_t = k; x_1, \dots, x_t)$: Vďaka Markovovej vlastnosti (budúcnosť závisí len od aktuálneho stavu, nie od minulosti) sa toto zjednoduší na $P(x_{t+1}, \dots, x_n | \pi_t = k)$, čo je presne to, čo počíta **Backward algoritmus** (skóre dozadu).

Preto sa v ďalšom kroku (8.9) tento výraz prepíše ako $f_k(t) \cdot b_k(t)$.

Celá „zložitosť“ výpočtu posteriórnej pravdepodobnosti sa týmto rozkladom rozdelila na dva samostatné, ľahko riešiteľné problémy.

5. Posterior Decoding

Prechod od vzťahu (2) k (3) je pre fungovanie HMM absolútne kľúčový. **Je to moment, kedy sa "zložité" pravdepodobnosti celého systému menia na "jednoduché" rekurzívne výpočty, ktoré vieme programovo riešiť.**

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k; x_1; \dots; x_t) \cdot P(x_{t+1}; \dots; x_n | \pi_t = k; x_1; \dots; x_t) \quad (2)$$

Pomocou Markovovej vlastnosti môžeme tento výraz jednoducho zapísať nasledovne:

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k; x_1; \dots; x_t) \cdot P(x_{t+1}; \dots; x_n | \pi_t = k) = \operatorname{argmax}_k f_k(t) \cdot b_k(t) \quad (3)$$

Prechod sa opiera o **Markovovu vlastnosť** a definíciu funkcií $f_k(t)$ a $b_k(t)$.

1. Rozbor vzťahu (2)

Vzťah (2) máme v tvare:

$$\pi_t^* = \operatorname{argmax}_k \left[\underbrace{P(\pi_t = k; x_1, \dots, x_t)}_{\text{časť A}} \cdot \underbrace{P(x_{t+1}, \dots, x_n | \pi_t = k; x_1, \dots, x_t)}_{\text{časť B}} \right]$$

5. Posterior Decoding

2. Aplikácia Markovovej vlastnosti na "časť B"

Toto je miesto, kde dochádza k "matematickej mágii". Pozrime sa na podmienenú pravdepodobnosť v časti B:

$$P(x_{t+1}, \dots, x_n \mid \pi_t = k; x_1, \dots, x_t)$$

Čo nám hovorí Markovova vlastnosť?

V HMM platí predpoklad, že budúcnosť závisí výlučne od aktuálneho stavu.

Ak poznáme stav v čase t ($\pi_t = k$), je úplne jedno, aká bola minulosť ($x_1, \dots, x_t$). Minulosť neobsahuje žiadnu ďalšiu informáciu, ktorá by ovplyvnila budúcnosť, ak už vieme, v akom stave sme teraz.

Matematicky to znamená, že podmienka na minulosť ($x_1, \dots, x_t$) je nadbytočná a môžeme ju odstrániť:

$$P(x_{t+1}, \dots, x_n \mid \pi_t = k; \underbrace{x_1, \dots, x_t}_{\text{minulosť}}) = P(x_{t+1}, \dots, x_n \mid \pi_t = k)$$

5. Posterior Decoding

3. Substitúcia definícií

Teraz už len dosadíme definície, ktoré sú štandardom pre tieto algoritmy:

- **Dopredná premenná (Forward):**

$$f_k(t) = P(\pi_t = k; x_1, \dots, x_t)$$

Toto presne zodpovedá Časti A vo vzťahu (2). Je to pravdepodobnosť, že sme v čase t v stave k a videli sme celú doterajšiu históriu.

- **Spätná premenná (Backward):**

$$b_k(t) = P(x_{t+1}, \dots, x_n | \pi_t = k)$$

Toto presne zodpovedá Časti B (po zjednodušení Markovovou vlastnosťou). Je to pravdepodobnosť, že uvidíme zvyšok sekvencie, ak sme v čase t v stave k .

4. Zostavenie vzťahu (3)

Keď nahradíme pôvodné zložité výrazy týmito definíciami, dostaneme:

$$\pi_t^* = \arg \max_k [f_k(t) \cdot b_k(t)]$$

Týmto rozkladom sme problém našej "najpravdepodobnejšej cesty" (posteriórneho dekódovania) rozbili na dva nezávislé podproblémy:

1. $f_k(t)$ vypočítame **Forward algoritmom** (číta dáta zľava doprava).
2. $b_k(t)$ vypočítame **Backward algoritmom** (číta dáta sprava doľava).

5. Posterior Decoding

4. Zostavenie vzťahu (3)

Keď nahradíme pôvodné zložité výrazy týmito definíciami, dostaneme:

$$\pi_t^* = \operatorname{argmax}_k [f_k(t) \cdot b_k(t)]$$

Týmto rozkladom sme problém našej "najpravdepodobnejšej cesty" (posteriórneho dekódovania) rozbili na dva nezávislé podproblémy:

1. $f_k(t)$ vypočítame **Forward algoritmom** (číta dáta zľava doprava).
2. $b_k(t)$ vypočítame **Backward algoritmom** (číta dáta sprava doľava).

Výsledkom je, že pre každý časový bod t a každý stav k jednoducho vynásobíme tieto dve hodnoty a vyberieme ten stav k , ktorý dáva najväčší súčin. Už nemusíme počítat obrovské pravdepodobnosti celých sekvencií naraz, ale len tieto lokálne súčiny.

Backward algoritmus

Ako bolo opísané predtým, backward (spätný) algoritmus sa používa na výpočet nasledujúcej pravdepodobnosti:

$$b_k(t) = P(x_{t+1}, \dots, x_n | \pi_t = k) \quad (4)$$

Rekurziu môžeme začať rozvíjať rozšírením do nasledujúceho tvaru:

$$b_k(t) = \sum_l P(x_{t+1}, \dots, x_n, \pi_{t+1} = l | \pi_t = k) \quad (5)$$

Z Markovovej vlastnosti potom získame:

$$b_k(t) = \sum_l P(x_{t+2}, \dots, x_n | \pi_{t+1} = l) \cdot P(\pi_{t+1} = l | \pi_t = k) \cdot P(x_{t+1} | \pi_{t+1} = l) \quad (6)$$

Prvý člen jednoducho zodpovedá $b_l(t+1)$. Vyjadrenie pomocou emisných a prechodových pravdepodobností dáva konečnú rekurziu:

$$b_k(t) = \sum_l b_l(t+1) \cdot a_{kl} \cdot e_l(x_{t+1}) \quad (7)$$

5. Posterior Decoding

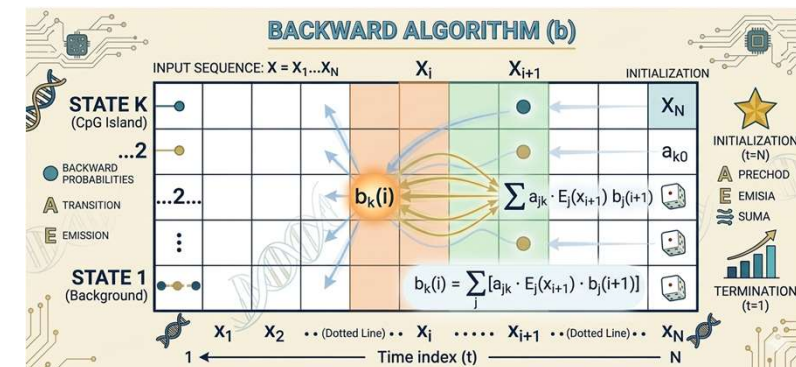
Porovnanie dopredných a spätných rekurzí vedie k zaujímavým poznatkom.

- **Zatiaľ čo dopredný (forward) algoritmus používa výsledky z $t-1$ na výpočet výsledku pre t , spätný (backward) algoritmus používa výsledky z $t+1$, čo prirodzene vedie k ich pomenovaniu.**
- **Ďalší významný rozdiel spočíva v emisných pravdepodobnostiach; zatiaľ čo emisie pre dopredný algoritmus vychádzajú z aktuálneho stavu a preto môžu byť vyňaté zo sumácie, emisie pre spätný algoritmus nastávajú v čase $t+1$ a preto musia byť zahrnuté v rámci sumácie.**

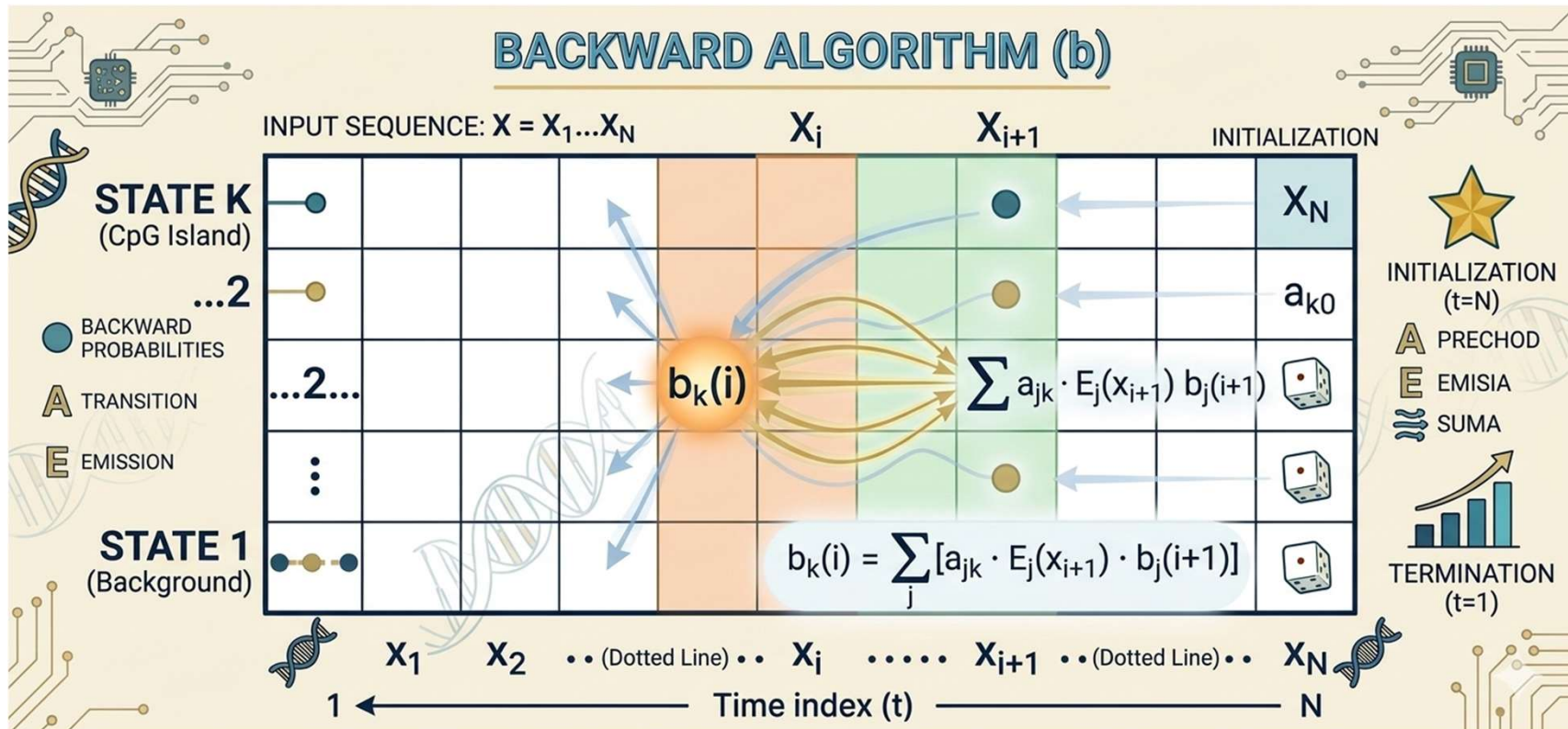
Vzhľadom na ich podobnosti nie je prekvapujúce, že spätný algoritmus je tiež implementovaný pomocou dynamickej programovacej tabuľky veľkosti $K \times N$.

Algoritmus, ako je znázornené na obrázku, začína inicializáciou najpravšieho stĺpca tabuľky na jednotku. Postupujúc sprava doľava, každý stĺpec sa potom vypočíta tak, že sa urobí vážený súčet hodnôt v stĺpci napravo podľa vyššie uvedenej rekurzcie. Po vypočítaní najľavejšieho stĺpca sú všetky spätné pravdepodobnosti vypočítané a algoritmus končí. Pretože existuje $K \cdot N$ záznamov a každý záznam skúma celkovo K ďalších záznamov, vedie to k časovej zložitosti $O(K^2 N)$ a priestorovej $O(K \cdot N)$, čo sú hranice totožné s dopredným algoritmom.

Rovnako ako $P(X)$ bolo vypočítané sčítaním najpravšieho stĺpca DP tabuľky dopredného algoritmu, $P(X)$ možno tiež vypočítať zo súčtu najľavejšieho stĺpca DP tabuľky spätného algoritmu. Preto sú tieto metódy pre tento konkrétny výpočet prakticky zameniteľné.



5. Posterior Decoding



5. Posterior Decoding

Podrobné odvedenie jednotlivých krokov

Od (4) k (5) (Zavedenie medzikroku)

Ako bolo opísané predtým, backward (spätný) algoritmus sa používa na výpočet nasledujúcej pravdepodobnosti:

$$b_k(t) = P(x_{t+1}, \dots, x_n | \pi_t = k) \quad (4)$$

Rekurziu môžeme začať rozvíjať rozšírením do nasledujúceho tvaru:

$$b_k(t) = \sum_l P(x_{t+1}, \dots, x_n, \pi_{t+1} = l | \pi_t = k) \quad (5)$$

Vzťah (4) hovorí, že $b_k(t)$ je pravdepodobnosť pozorovaní od $t+1$ do n , ak sme v čase t v stave k . Aby sme túto pravdepodobnosť rozložili, potrebujeme sa dostať do stavu v čase $t+1$.

Použijeme zákon celkovej pravdepodobnosti. Súčet pravdepodobností cez všetky možné stavy l , v ktorých sa môžeme nachádzať v čase $t+1$, sa musí rovnať celkovej pravdepodobnosti.

$$b_k(t) = \sum_l P(x_{t+1}, \dots, x_n, \pi_{t+1} = l | \pi_t = k)$$

Tým, že sme do podmienenej pravdepodobnosti pridali $\pi_{t+1} = l$, sme „otvorili cestu“ k rozkladu na jednotlivé kroky (prechod a emisiu).

5. Posterior Decoding

Od (5) k (6) (Aplikácia Markovovej vlastnosti)

Toto je najdôležitejší krok. Použijeme definíciu podmienenej pravdepodobnosti a Markovovu vlastnosť.

Rekurziu môžeme začať rozvíjať rozšírením do nasledujúceho tvaru:

$$b_k(t) = \sum_l P(x_{t+1}, \dots, x_n, \pi_{t+1} = l \mid \pi_t = k) \quad (5)$$

Z Markovovej vlastnosti potom získame:

$$b_k(t) = \sum_l P(x_{t+2}, \dots, x_n \mid \pi_{t+1} = l) \cdot P(\pi_{t+1} = l \mid \pi_t = k) \cdot P(x_{t+1} \mid \pi_{t+1} = l) \quad (6)$$

Pravdepodobnosť vo vnútri sumy v (5) môžeme pomocou reťazového pravidla (chain rule) rozdeliť na:

$$P(x_{t+1}, \dots, x_n, \pi_{t+1} = l \mid \pi_t = k) = P(x_{t+1}, \dots, x_n \mid \pi_{t+1} = l, \pi_t = k) \cdot P(\pi_{t+1} = l \mid \pi_t = k)$$

Teraz aplikujeme Markovovu vlastnosť: pravdepodobnosť budúcnosti závisí len od *aktuálneho* stavu ($\pi_{t+1} = l$), nie od stavu v minulosti ($\pi_t = k$). Preto $P(x_{t+1}, \dots, x_n \mid \pi_{t+1} = l, \pi_t = k)$ zjednodušíme na $P(x_{t+1}, \dots, x_n \mid \pi_{t+1} = l)$.

Teraz rozdelíme pozorovanú sekvenciu $x_{t+1}, \dots, x_n$ na prvý znak (x_{t+1}) a zvyšok ($x_{t+2}, \dots, x_n$):

- $P(x_{t+1}, \dots, x_n \mid \pi_{t+1} = l) = P(x_{t+1} \mid \pi_{t+1} = l) \cdot P(x_{t+2}, \dots, x_n \mid \pi_{t+1} = l)$

Keď to všetko dáme dokopy, dostaneme vzorec (6):

- $P(x_{t+2}, \dots, x_n \mid \pi_{t+1} = l)$ je "budúcnosť"
- $P(\pi_{t+1} = l \mid \pi_t = k)$ je prechod (transition)
- $P(x_{t+1} \mid \pi_{t+1} = l)$ je emisia (emission)

5. Posterior Decoding

Od (6) k (7) (Dosadenie premenných)

Z Markovovej vlastnosti potom získame:

$$b_k(t) = \sum_l P(x_{t+2}, \dots, x_n | \pi_{t+1} = l) \cdot P(\pi_{t+1} = l | \pi_t = k) \cdot P(x_{t+1} | \pi_{t+1} = l) \quad (6)$$

Prvý člen jednoducho zodpovedá $b_l(t+1)$. Vyjadrenie pomocou emisných a prechodových pravdepodobností dáva konečnú rekúriu:

$$b_k(t) = \sum_l b_l(t+1) \cdot a_{kl} \cdot e_l(x_{t+1}) \quad (7)$$

Toto je priama substitúcia podľa definícií modelu:

1. Pravdepodobnosť prechodu $P(\pi_{t+1} = l | \pi_t = k)$ je definovaná ako a_{kl} .
2. Pravdepodobnosť emisie $P(x_{t+1} | \pi_{t+1} = l)$ je definovaná ako $e_l(x_{t+1})$.
3. Pravdepodobnosť budúcnosti $P(x_{t+2}, \dots, x_n | \pi_{t+1} = l)$ je podľa definície (4) rovná $b_l(t+1)$.

Keď tieto symboly dosadíme do (6), dostaneme výslednú rekúriu (7):

$$b_k(t) = \sum_l a_{kl} \cdot e_l(x_{t+1}) \cdot b_l(t+1)$$

5. Posterior Decoding

Backward (spätný) algoritmus je "zrkadlovým obrazom" Forward algoritmu.

Zatiaľ čo Forward algoritmus zisťuje pravdepodobnosť sekvencie zľava doprava (od začiatku), **Backward algoritmus zisťuje pravdepodobnosť pozorovania zvyšku sekvencie sprava doľava (od konca).**

Príklad „Dehonested Casino“

V našom prípade máme sekvenciu $x = \{1, 6\}$, teda $N = 2$ pozorovania.

1. Parametre modelu (zhrnutie)

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacie pravdepodobnosti (π):** $\pi_F = 0,5$, $\pi_L = 0,5$
- **Matica prechodov (A):**
 - $P(F \rightarrow F) = 0,8$
 - $P(F \rightarrow L) = 0,2$
 - $P(L \rightarrow F) = 0,2$
 - $P(L \rightarrow L) = 0,8$
- **Emisné pravdepodobnosti (E):**
 - Pre F : $P(1|F) = 0,5$, $P(6|F) = 0,1$
 - Pre L : $P(1|L) = 0,1$, $P(6|L) = 0,5$

2. Krok: Inicializácia ($t = 2$)

Pri Backward algoritme začíname od posledného pozorovania (v našom prípade $t = 2$). Definujeme spätnú pravdepodobnosť $b_k(N)$ ako 1.

Prečo 1? Pretože pravdepodobnosť, že po poslednom pozorovaní už nevidíme žiadne ďalšie dáta, je 100% (teda 1).

- $b_F(2) = 1$
- $b_L(2) = 1$

5. Posterior Decoding

3. Krok: Rekurgia ($t = 1$)

Teraz vypočítame hodnoty pre čas $t = 1$. Hľadáme pravdepodobnosť, že ak sme v čase $t = 1$ v stave F (alebo L), uvidíme zvyšok sekvencie (v našom prípade hod 6).

Všeobecný vzorec je:

$$b_k(t) = \sum_{l \in \{F, L\}} A_{kl} \cdot E_l(x_{t+1}) \cdot b_l(t+1)$$

A) Výpočet pre $b_F(1)$ (Sme v stave Férová):

Sledujeme obe možné cesty, kam môžeme prejsť do času $t = 2$:

1. Cesta Férová $\rightarrow$ Férová:

- Pravdepodobnosť prechodu ($F \rightarrow F$): 0,8
- Emisia hodnoty 6 v stave F ($P(6|F)$): 0,1
- Spätná hodnota z času $t = 2$ ($b_F(2)$): 1
- Súčin: $0,8 \cdot 0,1 \cdot 1 = 0,08$

2. Cesta Férová $\rightarrow$ Falošná:

- Pravdepodobnosť prechodu ($F \rightarrow L$): 0,2
- Emisia hodnoty 6 v stave L ($P(6|L)$): 0,5
- Spätná hodnota z času $t = 2$ ($b_L(2)$): 1
- Súčin: $0,2 \cdot 0,5 \cdot 1 = 0,10$

Výsledok pre $b_F(1)$: Sčítame tieto cesty:

$$b_F(1) = 0,08 + 0,10 = \mathbf{0,18}$$

B) Výpočet pre $b_L(1)$ (Sme v stave Falošná):

Sledujeme obe možné cesty, kam môžeme prejsť do času $t = 2$:

1. Cesta Falošná $\rightarrow$ Férová:

- Pravdepodobnosť prechodu ($L \rightarrow F$): 0,2
- Emisia hodnoty 6 v stave F ($P(6|F)$): 0,1
- Spätná hodnota z času $t = 2$ ($b_F(2)$): 1
- Súčin: $0,2 \cdot 0,1 \cdot 1 = 0,02$

2. Cesta Falošná $\rightarrow$ Falošná:

- Pravdepodobnosť prechodu ($L \rightarrow L$): 0,8
- Emisia hodnoty 6 v stave L ($P(6|L)$): 0,5
- Spätná hodnota z času $t = 2$ ($b_L(2)$): 1
- Súčin: $0,8 \cdot 0,5 \cdot 1 = 0,40$

Výsledok pre $b_L(1)$: Sčítame tieto cesty:

$$b_L(1) = 0,02 + 0,40 = \mathbf{0,42}$$

5. Posterior Decoding

4. Krok: Ukončenie (Celková pravdepodobnosť $P(x)$)

Hoci sme vypočítali spätné pravdepodobnosti, cieľom je často zistiť celkovú pravdepodobnosť sekvencie $P(x)$. Na to potrebujeme skombinovať štartovacie pravdepodobnosti (π), prvú emisiu ($x_1 = 1$) a naše vypočítané spätné hodnoty ($b_k(1)$).

$$\text{Vzorec: } P(x) = \sum_{k \in \{F, L\}} \pi_k \cdot E_k(x_1) \cdot b_k(1)$$

1. Príspevok zo stavu F :

$$\bullet \pi_F \cdot P(1|F) \cdot b_F(1) = 0,5 \cdot 0,5 \cdot 0,18 = \mathbf{0,045}$$

2. Príspevok zo stavu L :

$$\bullet \pi_L \cdot P(1|L) \cdot b_L(1) = 0,5 \cdot 0,1 \cdot 0,42 = \mathbf{0,021}$$

Celková pravdepodobnosť sekvencie $\{1, 6\}$ je:

$$P(x) = 0,045 + 0,021 = \mathbf{0,066}$$

Čo nám tieto výsledky hovoria?

- Výsledok $b_F(1) = 0,18$ nám hovorí: "Ak sa v prvom kroku nachádzaš vo Férovom stave, existuje 18% šanca, že zvyšok sekvencie (hod 6) bude vygenerovaný správne podľa modelu."
- Výsledok $b_L(1) = 0,42$ nám hovorí: "Ak sa v prvom kroku nachádzaš vo Falošnom stave, existuje 42% šanca, že zvyšok sekvencie (hod 6) bude vygenerovaný správne."

Prečo je $b_L(1)$ vyššie?

Pretože Falošná kocka (Loaded) má oveľa vyššiu pravdepodobnosť hodiť 6 (0,5) ako Férová kocka (0,1). Preto ak sme v čase $t = 2$ "dostali" 6, model spätne vyhodnotil, že je oveľa pravdepodobnejšie, že sme boli v Falošnom stave.

5. Posterior Decoding

Pri adaptácii príkladu na sekvenciu troch hodov $\{1, 6, 6\}$ musíme algoritmus rozšíriť o jeden krok rekurzie navyše. Backward algoritmus začína od konca (od $t = 3$) a postupuje smerom k začiatku.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacie pravdepodobnosti:** $\pi_F = 0,5$, $\pi_L = 0,5$
- **Prechody (A):**
 - $F \rightarrow F = 0,8$, $F \rightarrow L = 0,2$
 - $L \rightarrow F = 0,2$, $L \rightarrow L = 0,8$
- **Emisie (E):**
 - $P(1|F) = 0,5$, $P(6|F) = 0,1$
 - $P(1|L) = 0,1$, $P(6|L) = 0,5$

Krok 1: Inicializácia ($t = 3$)

V čase $t = 3$ (posledný hod) definujeme, že pravdepodobnosť „zvyšku“ sekvencie je 1 (pretože za ním už nič nenasleduje).

- $b_F(3) = 1$
- $b_L(3) = 1$

Krok 2: Prvá rekurzia ($t = 2$, pozorovanie $x_3 = 6$)

Počítame pravdepodobnosť, že ak sme v čase $t = 2$ v danom stave, vyemitujeme 6 a následne skončíme.

$$\text{Vzorec: } b_k(2) = \sum_{l \in \{F, L\}} A_{kl} \cdot E_l(6) \cdot b_l(3)$$

• Výpočet pre $b_F(2)$:

$$b_F(2) = (A_{FF} \cdot E_F(6) \cdot b_F(3)) + (A_{FL} \cdot E_L(6) \cdot b_L(3))$$

$$b_F(2) = (0,8 \cdot 0,1 \cdot 1) + (0,2 \cdot 0,5 \cdot 1) = 0,08 + 0,10 = \mathbf{0,18}$$

• Výpočet pre $b_L(2)$:

$$b_L(2) = (A_{LF} \cdot E_F(6) \cdot b_F(3)) + (A_{LL} \cdot E_L(6) \cdot b_L(3))$$

$$b_L(2) = (0,2 \cdot 0,1 \cdot 1) + (0,8 \cdot 0,5 \cdot 1) = 0,02 + 0,40 = \mathbf{0,42}$$

Krok 3: Druhá rekurzia ($t = 1$, pozorovanie $x_2 = 6$)

Teraz využijeme výsledky z kroku 2 na výpočet pravdepodobnosti v čase $t = 1$.

$$\text{Vzorec: } b_k(1) = \sum_{l \in \{F, L\}} A_{kl} \cdot E_l(6) \cdot b_l(2)$$

• Výpočet pre $b_F(1)$:

$$b_F(1) = (A_{FF} \cdot E_F(6) \cdot b_F(2)) + (A_{FL} \cdot E_L(6) \cdot b_L(2))$$

$$b_F(1) = (0,8 \cdot 0,1 \cdot 0,18) + (0,2 \cdot 0,5 \cdot 0,42)$$

$$b_F(1) = (0,08 \cdot 0,18) + (0,1 \cdot 0,42) = 0,0144 + 0,042 = \mathbf{0,0564}$$

• Výpočet pre $b_L(1)$:

$$b_L(1) = (A_{LF} \cdot E_F(6) \cdot b_F(2)) + (A_{LL} \cdot E_L(6) \cdot b_L(2))$$

$$b_L(1) = (0,2 \cdot 0,1 \cdot 0,18) + (0,8 \cdot 0,5 \cdot 0,42)$$

$$b_L(1) = (0,02 \cdot 0,18) + (0,4 \cdot 0,42) = 0,0036 + 0,168 = \mathbf{0,1716}$$

5. Posterior Decoding

Krok 4: Ukončenie (Celková pravdepodobnosť $P(x)$)

Teraz vypočítame celkovú pravdepodobnosť, že model vygeneroval sekvenciu $\{1, 6, 6\}$.

Použijeme počiatočné pravdepodobnosti (π) a prvý pozorovaný znak $x_1 = 1$.

$$\text{Vzorec: } P(x) = \sum_{k \in \{F, L\}} \pi_k \cdot E_k(1) \cdot b_k(1)$$

- **Príspevok zo stavu F :**

$$\pi_F \cdot E_F(1) \cdot b_F(1) = 0,5 \cdot 0,5 \cdot 0,0564 = 0,25 \cdot 0,0564 = 0,0141$$

- **Príspevok zo stavu L :**

$$\pi_L \cdot E_L(1) \cdot b_L(1) = 0,5 \cdot 0,1 \cdot 0,1716 = 0,05 \cdot 0,1716 = 0,00858$$

Výsledná pravdepodobnosť:

$$P(x) = 0,0141 + 0,00858 = \mathbf{0,02268}$$

Záver: Celková pravdepodobnosť pozorovanej sekvencie $\{1, 6, 6\}$ v tomto modeli je **2,268 %**.

5. Posterior Decoding

Na to, aby sme naplno pochopili silu HMM (Skrytých Markovových modelov), je najlepšie ukázať, ako **Forward** a **Backward** algoritmy spolupracujú. Ich spojením získame tzv.

Posteriórne dekódovanie – teda schopnosť s vysokou presnosťou určiť, v akom stave sa systém nachádzal v každom časovom bode, keď poznáme celú sekvenciu dát.

Použijeme rovnaký model "Mini Kasíno" a sekvenciu $x = \{1, 6\}$.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
 - **Štartovacia matica (π):** $\pi_F = 0,5, \pi_L = 0,5$
 - **Prechody (A):** $F \rightarrow F = 0,8; F \rightarrow L = 0,2; L \rightarrow F = 0,2; L \rightarrow L = 0,8$
 - **Emisie (E):** $E_F(1) = 0,5, E_F(6) = 0,1; E_L(1) = 0,1, E_L(6) = 0,5$
-

5. Posterior Decoding

Na to, aby sme naplno pochopili silu HMM (Skrytých Markovových modelov), je najlepšie ukázať, ako **Forward** a **Backward** algoritmy spolupracujú. Ich spojením získame tzv.

Posteriórne dekodovanie – teda schopnosť s vysokou presnosťou určiť, v akom stave sa systém nachádzal v každom časovom bode, keď poznáme celú sekvenciu dát.

Použijeme rovnaký model "Mini Kasíno" a sekvenciu $x = \{1, 6\}$.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacia matica (π):** $\pi_F = 0,5, \pi_L = 0,5$
- **Prechody (A):** $F \rightarrow F = 0,8; F \rightarrow L = 0,2; L \rightarrow F = 0,2; L \rightarrow L = 0,8$
- **Emisie (E):** $E_F(1) = 0,5, E_F(6) = 0,1; E_L(1) = 0,1, E_L(6) = 0,5$

1. Forward algoritmus (Dopredný - zľava doprava)

Tento algoritmus nám povie, aká je pravdepodobnosť, že sme v stave k v čase t , **po prečítaní doterajšej histórie**.

Krok 1: Inicializácia ($t = 1$, hodnota 1)

- $f_F(1) = \pi_F \cdot E_F(1) = 0,5 \cdot 0,5 = \mathbf{0,25}$
- $f_L(1) = \pi_L \cdot E_L(1) = 0,5 \cdot 0,1 = \mathbf{0,05}$

Krok 2: Rekúzia ($t = 2$, hodnota 6)

- $f_F(2) = [f_F(1) \cdot A_{FF} + f_L(1) \cdot A_{LF}] \cdot E_F(6)$
 - $f_F(2) = [0,25 \cdot 0,8 + 0,05 \cdot 0,2] \cdot 0,1 = [0,20 + 0,01] \cdot 0,1 = \mathbf{0,021}$
- $f_L(2) = [f_F(1) \cdot A_{FL} + f_L(1) \cdot A_{LL}] \cdot E_L(6)$
 - $f_L(2) = [0,25 \cdot 0,2 + 0,05 \cdot 0,8] \cdot 0,5 = [0,05 + 0,04] \cdot 0,5 = \mathbf{0,045}$

5. Posterior Decoding

Na to, aby sme naplno pochopili silu HMM (Skrytých Markovových modelov), je najlepšie ukázať, ako **Forward** a **Backward** algoritmy spolupracujú. Ich spojením získame tzv.

Posteriórne dekodovanie – teda schopnosť s vysokou presnosťou určiť, v akom stave sa systém nachádzal v každom časovom bode, keď poznáme celú sekvenciu dát.

Použijeme rovnaký model "Mini Kasíno" a sekvenciu $x = \{1, 6\}$.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacia matica (π):** $\pi_F = 0,5, \pi_L = 0,5$
- **Prechody (A):** $F \rightarrow F = 0,8; F \rightarrow L = 0,2; L \rightarrow F = 0,2; L \rightarrow L = 0,8$
- **Emisie (E):** $E_F(1) = 0,5, E_F(6) = 0,1; E_L(1) = 0,1, E_L(6) = 0,5$

2. Backward algoritmus (Spätný - sprava doľava)

Tento algoritmus nám povie, aká je pravdepodobnosť, že uvidíme zvyšok sekvencie, **ak sme teraz v stave k** .

Krok 1: Inicializácia ($t = 2$)

- $b_F(2) = 1$
- $b_L(2) = 1$

Krok 2: Rekúzia ($t = 1$, pozorujeme 6)

- $b_F(1) = A_{FF} \cdot E_F(6) \cdot b_F(2) + A_{FL} \cdot E_L(6) \cdot b_L(2)$
 - $b_F(1) = 0,8 \cdot 0,1 \cdot 1 + 0,2 \cdot 0,5 \cdot 1 = 0,08 + 0,1 = \mathbf{0,18}$
- $b_L(1) = A_{LF} \cdot E_F(6) \cdot b_F(2) + A_{LL} \cdot E_L(6) \cdot b_L(2)$
 - $b_L(1) = 0,2 \cdot 0,1 \cdot 1 + 0,8 \cdot 0,5 \cdot 1 = 0,02 + 0,4 = \mathbf{0,42}$

5. Posterior Decoding

3. Integrácia: Posteriórne dekodovanie

Teraz urobíme to najdôležitejšie. Pre každý čas t a stav k vynásobíme Forward výsledok so Backward výsledkom. To nám dá pravdepodobnosť, že sme v čase t v stave k **vzhľadom na celú pozorovanú sekvenciu**.

Definujme posteriórnu pravdepodobnosť $\gamma_k(t)$ ako:

$$\gamma_k(t) = \frac{f_k(t) \cdot b_k(t)}{P(x)}$$

(Kde $P(x)$ je celková pravdepodobnosť sekvencie, čo je súčet $f_F(2) + f_L(2) = 0,021 + 0,045 = 0,066$)

Výpočet pre čas $t = 1$:

- **Pravdepodobnosť stavu Férová (F):**

$$\gamma_F(1) = \frac{f_F(1) \cdot b_F(1)}{P(x)} = \frac{0,25 \cdot 0,18}{0,066} = \frac{0,045}{0,066} \approx \mathbf{0,682}$$

- **Pravdepodobnosť stavu Falošná (L):**

$$\gamma_L(1) = \frac{f_L(1) \cdot b_L(1)}{P(x)} = \frac{0,05 \cdot 0,42}{0,066} = \frac{0,021}{0,066} \approx \mathbf{0,318}$$

Záver: Čo sme zistili?

V čase $t = 1$ (keď padla 1), nám model po zohľadnení celej sekvencie $\{1, 6\}$ hovorí:

- Existuje **68,2 % šanca**, že kocka bola **Férová**.
- Existuje **31,8 % šanca**, že kocka bola **Falošná**.

Bez Backward algoritmu (teda iba pri pohľade na prvý hod 1) by sme mali iba pravdepodobnosti 0,25 vs 0,05 (normalizované na 83% vs 17%). Tým, že sme použili **Backward algoritmus**, sme "získali informáciu z budúcnosti" (z druhého hodu, kde padla 6), čo nám umožnilo spresniť náš odhad pre prvý hod. Toto je podstata posteriórneho dekodovania.

5. Posterior Decoding

Model mini kasína pre tri pozorovania

Pri rozšírení na tri pozorovania $\{1, 6, 6\}$ sa algoritmus stáva komplexnejším, ale princíp zostáva rovnaký. Teraz musíme vykonať viac krokov rekurzcie, aby sme spracovali celú sekvenciu.

Tento proces je vynikajúcim príkladom toho, ako model "učí" o stave systému na základe ďalších dát.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacia matica (π):** $\pi_F = 0,5, \pi_L = 0,5$
- **Prechody (A):** $F \rightarrow F = 0,8; F \rightarrow L = 0,2; L \rightarrow F = 0,2; L \rightarrow L = 0,8$
- **Emisie (E):** $E_F(1) = 0,5, E_F(6) = 0,1; E_L(1) = 0,1, E_L(6) = 0,5$

1. Forward Algoritmus (Dopredný)

Počítame pravdepodobnosti $f_k(t)$, ktoré vyjadrujú: "Aká je šanca, že sme v stave k v čase t a videli sme všetky dáta doteraz?"

Krok 1 ($t=1, x=1$):

- $f_F(1) = \pi_F \cdot E_F(1) = 0,5 \cdot 0,5 = \mathbf{0,25}$
- $f_L(1) = \pi_L \cdot E_L(1) = 0,5 \cdot 0,1 = \mathbf{0,05}$

Krok 2 ($t=2, x=6$):

- $f_F(2) = [f_F(1) \cdot A_{FF} + f_L(1) \cdot A_{LF}] \cdot E_F(6) = [0,25 \cdot 0,8 + 0,05 \cdot 0,2] \cdot 0,1 = \mathbf{0,021}$
- $f_L(2) = [f_F(1) \cdot A_{FL} + f_L(1) \cdot A_{LL}] \cdot E_L(6) = [0,25 \cdot 0,2 + 0,05 \cdot 0,8] \cdot 0,5 = \mathbf{0,045}$

Krok 3 ($t=3, x=6$):

- $f_F(3) = [f_F(2) \cdot A_{FF} + f_L(2) \cdot A_{LF}] \cdot E_F(6) = [0,021 \cdot 0,8 + 0,045 \cdot 0,2] \cdot 0,1 = (0,0168 + 0,009) \cdot 0,1 = \mathbf{0,00258}$
- $f_L(3) = [f_F(2) \cdot A_{FL} + f_L(2) \cdot A_{LL}] \cdot E_L(6) = [0,021 \cdot 0,2 + 0,045 \cdot 0,8] \cdot 0,5 = (0,0042 + 0,036) \cdot 0,5 = \mathbf{0,0201}$

Celková pravdepodobnosť $P(X) = 0,00258 + 0,0201 = \mathbf{0,02268}$

5. Posterior Decoding

Model mini kasína pre tri pozorovania

Pri rozšírení na tri pozorovania $\{1, 6, 6\}$ sa algoritmus stáva komplexnejším, ale princíp zostáva rovnaký. Teraz musíme vykonať viac krokov rekúzie, aby sme spracovali celú sekvenciu.

Tento proces je vynikajúcim príkladom toho, ako model "učí" o stave systému na základe ďalších dát.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacia matica (π):** $\pi_F = 0,5, \pi_L = 0,5$
- **Prechody (A):** $F \rightarrow F = 0,8; F \rightarrow L = 0,2; L \rightarrow F = 0,2; L \rightarrow L = 0,8$
- **Emisie (E):** $E_F(1) = 0,5, E_F(6) = 0,1; E_L(1) = 0,1, E_L(6) = 0,5$

2. Backward Algoritmus (Spätný)

Počítame $b_k(t)$, ktoré vyjadrujú: "Aká je šanca, že uvidíme zvyšok sekvencie (6, 6), ak sme teraz v stave k ?"

Krok 1 (t=3):

- $b_F(3) = 1$
- $b_L(3) = 1$

Krok 2 (t=2, x=6):

- $b_F(2) = A_{FF} \cdot E_F(6) \cdot b_F(3) + A_{FL} \cdot E_L(6) \cdot b_L(3) = 0,8 \cdot 0,1 \cdot 1 + 0,2 \cdot 0,5 \cdot 1 = \mathbf{0,18}$
- $b_L(2) = A_{LF} \cdot E_F(6) \cdot b_F(3) + A_{LL} \cdot E_L(6) \cdot b_L(3) = 0,2 \cdot 0,1 \cdot 1 + 0,8 \cdot 0,5 \cdot 1 = \mathbf{0,42}$

Krok 3 (t=1, x=6):

- $b_F(1) = A_{FF} \cdot E_F(6) \cdot b_F(2) + A_{FL} \cdot E_L(6) \cdot b_L(2) = 0,8 \cdot 0,1 \cdot 0,18 + 0,2 \cdot 0,5 \cdot 0,42 = 0,0144 + 0,042 = \mathbf{0,0564}$
- $b_L(1) = A_{LF} \cdot E_F(6) \cdot b_F(2) + A_{LL} \cdot E_L(6) \cdot b_L(2) = 0,2 \cdot 0,1 \cdot 0,18 + 0,8 \cdot 0,5 \cdot 0,42 = 0,0036 + 0,168 = \mathbf{0,1716}$

5. Posterior Decoding

Model mini kasína pre tri pozorovania

Pri rozšírení na tri pozorovania $\{1, 6, 6\}$ sa algoritmus stáva komplexnejším, ale princíp zostáva rovnaký. Teraz musíme vykonať viac krokov rekurzcie, aby sme spracovali celú sekvenciu.

Tento proces je vynikajúcim príkladom toho, ako model "učí" o stave systému na základe ďalších dát.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Štartovacia matica (π):** $\pi_F = 0,5, \pi_L = 0,5$
- **Prechody (A):** $F \rightarrow F = 0,8; F \rightarrow L = 0,2; L \rightarrow F = 0,2; L \rightarrow L = 0,8$
- **Emisie (E):** $E_F(1) = 0,5, E_F(6) = 0,1; E_L(1) = 0,1, E_L(6) = 0,5$

3. Posteriórne dekódovanie ($\gamma_k(t)$)

Teraz vypočítame pravdepodobnosť stavu v každom časovom bode, ak poznáme celú sekvenciu: $\gamma_k(t) = \frac{f_k(t) \cdot b_k(t)}{P(X)}$.

Čas (t)	Pozorovanie	Pravdepodobnosť Férová (γ_F)	Pravdepodobnosť Falošná (γ_L)
1	1	$\frac{0,25 \cdot 0,0564}{0,02268} \approx 62,2\%$	$\frac{0,05 \cdot 0,1716}{0,02268} \approx 37,8\%$
2	6	$\frac{0,021 \cdot 0,18}{0,02268} \approx 16,7\%$	$\frac{0,045 \cdot 0,42}{0,02268} \approx 83,3\%$
3	6	$\frac{0,00258 \cdot 1}{0,02268} \approx 11,4\%$	$\frac{0,0201 \cdot 1}{0,02268} \approx 88,6\%$

Interpretácia výsledkov

Všimnite si, ako sa naše vnímanie reality mení s prichádzajúcimi dátami:

1. **V čase $t = 1$:** Pozorujeme "1". Férová kocka ju hádže často, takže máme vysokú istotu (62%), že sme vo Férovom stave.
2. **V čase $t = 2$:** Pozorujeme "6". Model okamžite "prehodnotil" situáciu. Aj keď sme v $t = 1$ verili, že sme vo Férovom stave, príchod šestky spätne ovplyvnil náš úsudok o minulosti. V čase $t = 2$ je už pravdepodobnosť Falošnej kocky 83%.
3. **V čase $t = 3$:** Pozorujeme druhú "6". Dôvera v to, že kocka je Falošná, ešte viac vzrástla na 88,6%.

Tento príklad dokonale ilustruje, prečo je **Posteriórne dekódovanie** také výkonné: nepozera sa len na minulosť, ale kombinuje informácie z oboch smerov, aby nám podalo najpresnejší možný odhad stavu.

Učenie

Videli sme, ako bodovať a dekodovať sekvenciu generovanú HMM dvoma rôznymi spôsobmi. **Tieto metódy však predpokladali, že už poznáme emisné a prechodové pravdepodobnosti.** Hoci máme vždy možnosť tieto pravdepodobnosti odhadnúť, niekedy možno budeme chcieť použiť skôr empirický prístup založený na dátach na odvodenie týchto parametrov. **Našťastie, rámec HMM umožňuje učenie týchto pravdepodobností, ak máme k dispozícii sadu trénovacích dát a stanovenú architektúru modelu.**

Keď sú trénovacie dáta označené (labeled), odhad pravdepodobností je formou učenia s učiteľom (supervised learning). Jeden taký prípad by nastal, ak by sme dostali sekvenciu DNA dlhú milión nukleotidov, v ktorej boli všetky CpG ostrovčeky experimentálne anotované, a mali by sme to použiť na odhad parametrov nášho modelu.

Naopak, keď trénovacie dáta nie sú označené (unlabeled), problém odhadu je formou učenia bez učiteľa (unsupervised learning). Pokračujúc v príklade s CpG ostrovčekmi, táto situácia by nastala, ak by poskytnutá sekvencia DNA neobsahovala vôbec žiadnu anotáciu ostrovčekov a my by sme potrebovali odhadnúť parametre modelu a zároveň identifikovať ostrovčeky.

Učenie s učiteľom

Pri poskytnutí označených dát je myšlienka odhadu parametrov modelu priamočiara.

Predpokladajme, že dostanete označenú sekvenciu $x_1, \dots, x_N$ a tiež skutočnú sekvenciu skrytých stavov $\pi_1, \dots, \pi_N$. Intuitívne by sa dalo očakávať, že **pravdepodobnosti, ktoré maximalizujú vierohodnosť (likelihood) dát, sú skutočné pravdepodobnosti, ktoré pozorujeme v dátach.**

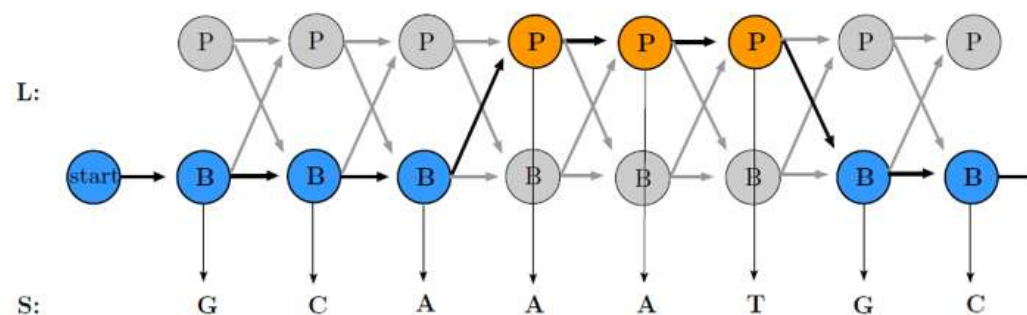
Je to skutočne tak a možno to formalizovať definovaním A_{kl} ako počet výskytov, kedy skrytý stav k prechádza do l , a $E_k(b)$ ako počet výskytov, kedy je znak b emitovaný zo skrytého stavu k . Parametre θ , ktoré maximalizujú $P(x|\theta)$, sa jednoducho získajú počítaním nasledovne:

$$a_{kl} = \frac{A_{kl}}{\sum_i A_{ki}} \quad (a)$$

$$e_k(b) = \frac{E_k(b)}{\sum_c E_k(c)} \quad (b)$$

5. Posterior Decoding

Príklad trénovacej množiny je zobrazený na obrázku.



Supervised Learning of CpG islands

V tomto príklade je zrejmé, že pravdepodobnosť prechodu z B do P je $\frac{1}{3+1} = \frac{1}{4}$ (sú tam 3 prechody z B do B a 1 prechod z B do P) a pravdepodobnosť emitovania G zo stavu B je $\frac{2}{2+2+1} = \frac{2}{5}$ (zo stavu B sú emitované 2 G-čky, 2 C-čky a 1 A-čko).

Všimnite si však, že vo vyššie uvedenom príklade je emisná pravdepodobnosť znaku T zo stavu B rovná 0, pretože v trénovacej množine sa takéto emisie nevyskytli. **Nulová pravdepodobnosť, či už pre prechod alebo emisiu, je obzvlášť problematická**, pretože vedie k nekonečnej logaritmickej penalizácii. **V skutočnosti však nulová pravdepodobnosť mohla vzniknúť iba v dôsledku nadmerného prispôbenia (over-fitting) alebo malej veľkosti vzorky.**

5. Posterior Decoding

Na nápravu tohto problému a zachovanie flexibility v našom modeli môžeme zbierať viac dát, na ktorých budeme trénovať, čím znížime možnosť, že nulová pravdepodobnosť je spôsobená malou veľkosťou vzorky.

Ďalšou možnosťou je použiť "pseudopočty" (pseudocounts) namiesto absolútnych počtov: umelé pridanie určitého počtu počtov k našim trénovacím dátam, o ktorých si myslíme, že presnejšie reprezentujú skutočné parametre a pomáhajú vykompenzovať chyby veľkosti vzorky.

- $A_{jd}^* = A_{jd} + r_{jd}$
- $E_k(b)^* = E_k(b) + r_k(b)$

Väčšie parametre pseudopočtov zodpovedajú silnému apriórному presvedčeniu o parametroch, čo sa odráža v skutočnosti, že tieto pseudopočty, odvodené z vašich apriórnych hodnôt, relatívne prevažujú nad pozorovaniami, vašimi trénovacími dátami. Rovnako tak sa malé parametre pseudopočtov ($r \ll 1$) častejšie používajú, keď sú naše apriorne informácie relatívne slabé a naším cieľom nie je prevažovať empirické dáta, ale len vyhnúť sa príliš drsným nulovým pravdepodobnostiam.

Učenie bez učiteľa

Učenie bez učiteľa zahŕňa odhadovanie parametrov na základe neoznačených dát. Môže sa to zdať nemožné – ako môžeme vziať dáta, o ktorých nevieme nič, a použiť ich na "učenie"? – ale iteratívny prístup môže priniesť prekvapivo dobré výsledky a je v týchto prípadoch typickou voľbou.

Dá sa to voľne prirovnať k evolučnému algoritmu: z určitej počiatočnej voľby parametrov algoritmus posúdi, ako dobre parametre vysvetľujú alebo súvisia s dátami, použije nejaký krok v tomto posúdení na zlepšenie parametrov a potom posúdi nové parametre, pričom v každom kroku produkuje prírastkové zlepšenia v parametroch, rovnako ako zdatnosť alebo jej nedostatok konkrétneho organizmu v jeho prostredí produkuje prírastkové nárasty v priebehu evolučného času, keď sú výhodné alely prednostne odovzdávané ďalej.

Predpokladajme, že máme určité apriórne presvedčenie o tom, aká by mala byť každá emisná a prechodová pravdepodobnosť. Vzhľadom na tieto parametre **môžeme použiť dekodovaciu metódu** na vyvodenie skrytých stavov podkladajúcich poskytnutú sekvenciu dát. Pomocou tohto konkrétneho dekodovacieho rozboru môžeme **následne pre-odhadnúť prechodové a emisné počty a pravdepodobnosti v procese podobnom tomu**, ktorý sa používa pri učení s učiteľom. Ak budeme tento postup opakovať, kým zlepšenie vierohodnosti dát zostane relatívne stabilné, sekvencia dát by mala v konečnom dôsledku nasmerovať parametre k ich vhodným hodnotám.

Expectation Maximization (EM) s využitím Viterbiho tréovania

Prvá metóda, Viterbiho tréovanie, je pomerne jednoduchá, ale nie úplne rigorózna. Po zvolení počiatočných najlepších odhadov parametrov modelu postupuje nasledovne:

1. **E krok (Expectation step):** Vykonajte Viterbiho dekodovanie, aby ste našli π^* , najpravdepodobnejšiu cestu stavov. Vypočítajte A_{ij}^* a $E_k(b)^*$ pomocou pseudopočtov založených na prechodoch a emisiách pozorovaných v stavoch π^* pri daných najnovších parametroch a pozorovanej sekvencii.
2. **M krok (Maximization step):** Vypočítajte nové parametre a_{ij} , $e_k(b)$ pomocou jednoduchého formálneho počítania ako pri učení s učiteľom.
3. **Iterácia:** Opakujte kroky E a M, kým vierohodnosť $P(x|\theta)$ nekonverguje.

Hoci Viterbiho tréovanie konverguje rýchlo, výsledné odhady parametrov sú zvyčajne horšie ako tie z Baum-Welchovho algoritmu. **Tento výsledok vyplýva zo skutočnosti, že Viterbiho tréovanie berie do úvahy iba najpravdepodobnejšiu skrytú cestu namiesto súboru všetkých možných skrytých ciest.**

5. Posterior Decoding

Expectation Maximization (EM): Baum-Welchov algoritmus

Rigoróznější přístup k učení bez učitele zahrňuje aplikaci Expectation Maximization na HMM. Vo všeobecnosti EM prebieha nasledovným spôsobom:

- **Inicializácia:** Inicializujte parametre na stav najlepšieho odhadu.
- **E krok:** Odhadnite očakávanú pravdepodobnosť skrytých stavov pri daných najnovších parametroch a pozorovanej sekvencii.
- **M krok:** Zvoľte nové parametre maximálnej vierohodnosti pomocou pravdepodobnostného rozdelenia skrytých stavov.
- **Iterácia:** Opakujte kroky E a M, kým vierohodnosť dát pri daných parametroch nekonverguje.

Sila EM spočíva v skutočnosti, že $P(x|\theta)$ sa zaručene zvyšuje s každou iteráciou algoritmu. Preto, keď táto pravdepodobnosť konverguje, dosiahlo sa lokálne maximum. V dôsledku toho, ak využijeme rôzne inicializačné stavy, budeme s najväčšou pravdepodobnosťou schopní identifikovať globálne maximum, t. j. najlepšie parametre θ . Baum-Welchov algoritmus zovšeobecňuje EM na HMM. Konkrétne používa dopredný (forward) a spätný (backward) algoritmus na výpočet $P(x|\theta)$ a na odhad A_{ij} a $E_k(b)$.

5. Posterior Decoding

Algoritmus postupuje nasledovne:

1. **Inicializácia:** Inicializujte parametre na stav najlepšieho odhadu.
2. **Iterácia:**
 1. Spustite dopredný (forward) algoritmus.
 2. Spustite spätný (backward) algoritmus.
 3. Vypočítajte novú logaritmickú vierohodnosť $P(x|\theta)$.
 4. Vypočítajte A_{kl} a $E_k(b)$.
 5. Vypočítajte a_{kl} a $e_k(b)$ pomocou vzorcov pre pseudopočty.
 6. Opakujte, kým $P(x|\theta)$ nekonverguje.

Predtým sme diskutovali o tom, ako vypočítať $P(x|\theta)$ pomocou konečných výsledkov dopredného alebo spätného algoritmu. Ale ako odhadneme A_{kl} a $E_k(b)$? Uvažujme o očakávanom počte prechodov zo stavu k do stavu l pri danej súčasnej sade parametrov θ . Toto očakávanie môžeme vyjadriť ako:

$$A_{kl} = \sum_t P(\pi_t = k; \pi_{t+1} = l | x; \theta) = \sum_t \frac{P(\pi_t = k; \pi_{t+1} = l; x | \theta)}{P(x | \theta)}$$

Využitie Markovovej vlastnosti a definícií emisných a prechodových pravdepodobností vedie k nasledujúcemu odvodeniu:

$$A_{kl} = \sum_t \frac{P(x_1 \dots x_t; \pi_t = k; \pi_{t+1} = l; x_{t+1} \dots x_N | \theta)}{P(x | \theta)} \quad (a)$$

$$= \sum_t \frac{P(x_1 \dots x_t; \pi_t = k) \cdot P(\pi_{t+1} = l; x_{t+1} \dots x_N | \pi_t; \theta)}{P(x | \theta)} \quad (b)$$

$$= \sum_t \frac{f_k(t) \cdot P(\pi_{t+1} = l | \pi_t = k) \cdot P(x_{t+1} | \pi_{t+1} = l) \cdot P(x_{t+2} \dots x_N | \pi_{t+1} = l; \theta)}{P(x | \theta)} \quad (c)$$

$$\Rightarrow A_{kl} = \sum_t \frac{f_k(t) \cdot a_{kl} \cdot e_l(x_{t+1}) \cdot b_l(t+1)}{P(x | \theta)} \quad (d)$$

Podobné odvodenie vedie k nasledujúcemu výrazu pre $E_k(b)$:

$$E_k(b) = \sum_{t|x_t=b} \frac{f_k(t) \cdot b_k(t)}{P(x | \theta)}$$

Preto spustením dopredného a spätného algoritmu máme všetky informácie potrebné na výpočet $P(x|\theta)$ a na aktualizáciu emisných a prechodových pravdepodobností počas každej iterácie. Pretože tieto aktualizácie sú operácie v konštantnom čase po tom, čo boli vypočítané $P(x|\theta)$, $f_k(t)$ a $b_k(t)$, celková časová zložitosť pre túto verziu učenia bez učiteľa je $\Theta(K^2NS)$, kde S je celkový počet iterácií.

5. Posterior Decoding

Model mini kasína

Expectation Maximization (EM) s využitím Viterbiho tréningu

Viterbiho tréning (často nazývané aj "Hard EM") je zaujímavý spôsob, ako učiť model bez toho, aby sme poznali skryté stavy. Namiesto toho, aby sme brali do úvahy všetky možné cesty (ako v Baum-Welchovom algoritme), vyberieme tú **jedinú najpravdepodobnejšiu cestu** a budeme sa tváriť, že toto je "pravda".

Podme si to ukázať na príklade.

1. Nastavenie príkladu

Máme model "Mini Kasína" (stavy F, L) a pozorovanú sekvenciu $x = \{1, 6\}$.

Keďže je to učenie bez učiteľa, **nepoznáme** skutočné pravdepodobnosti. Musíme začať s "naivným" odhadom (počiatočné parametre), ktoré budeme postupne vylepšovať.

Naše naivné počiatočné parametre (θ_{init}):

- **Prechody (A):** Predpokladajme, že všetko je rovnako pravdepodobné (0, 5).
 - $A_{FF} = 0,5, A_{FL} = 0,5, A_{LF} = 0,5, A_{LL} = 0,5$
- **Emisie (E):** Predpokladajme, že obe kocky sa správajú rovnako (nemáme o nich vedomosť).
 - $E_F(1) = 0,5, E_F(6) = 0,5$
 - $E_L(1) = 0,5, E_L(6) = 0,5$
- **Štartovacia matica:** $\pi_F = 0,5, \pi_L = 0,5$

2. Iterácia 1

E-Krok (Expectation Step): Hľadáme najlepšiu cestu

V tomto kroku použijeme Viterbiho algoritmus s našimi aktuálnymi parametrami, aby sme našli najpravdepodobnejšiu cestu $\pi^* = \{\pi_1, \pi_2\}$.

- **V čase $t = 1$ (pozorujeme 1):**

- $\delta_F(1) = \pi_F \cdot E_F(1) = 0,5 \cdot 0,5 = 0,25$
- $\delta_L(1) = \pi_L \cdot E_L(1) = 0,5 \cdot 0,5 = 0,25$
(V tomto bode je to remíza, obe cesty sú rovnako pravdepodobné.)

- **V čase $t = 2$ (pozorujeme 6):**

- $\delta_F(2) = \max(\delta_F(1) \cdot A_{FF}, \delta_L(1) \cdot A_{LF}) \cdot E_F(6)$
 - $\delta_F(2) = \max(0,25 \cdot 0,5, 0,25 \cdot 0,5) \cdot 0,5 = 0,125 \cdot 0,5 = 0,0625$
- $\delta_L(2) = \max(\delta_F(1) \cdot A_{FL}, \delta_L(1) \cdot A_{LL}) \cdot E_L(6)$
 - $\delta_L(2) = \max(0,25 \cdot 0,5, 0,25 \cdot 0,5) \cdot 0,5 = 0,125 \cdot 0,5 = 0,0625$

Výsledok E-kroku: Viterbi nám vráti cestu. Povedzme, že kvôli numerickej zhode algoritmus vybral cestu $F \rightarrow L$.

Naša "pseudo-pravda" pre túto iteráciu je: V stave 1 sme boli v F , v stave 2 sme boli v L .

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM) s využitím Viterbiho tréningu

M-Krok (Maximization Step): Aktualizujeme model

Teraz upravíme naše parametre tak, aby lepšie sedeli na túto "pseudo-pravdu" ($\pi_1 = F, \pi_2 = L$).

1. Počítame prechody (A):

- V ceste $F \rightarrow L$ vidíme 1 prechod z $F \rightarrow L$.
- Nové počty: $A_{FL} = 1$ (ostatné sú 0).
- Nové pravdepodobnosti (po normalizácii): $A_{FL} = 1, 0$ (a $A_{FF} = 0$).

2. Počítame emisie (E):

- V stave F sme emitovali 1 (count=1).
- V stave L sme emitovali 6 (count=1).
- Nové parametre: $E_F(1) = 1, 0, E_L(6) = 1, 0$.

Poznámka: V praxi by sme použili pseudopočty (pridali malé číslo, napr. 0,1), aby sme sa vyhli nulám.

3. Iterácia 2

Teraz máme "lepšie" parametre:

- $A_{FL} = 1, 0$
- $E_F(1) = 1, 0, E_L(6) = 1, 0$

Opakujeme E-krok:

S týmito novými parametrami spustíme Viterbi znova na tej istej sekvencii $\{1, 6\}$.

- $t = 1$: $\delta_F(1) = 0, 5 \cdot 1, 0 = 0, 5$ (Stav L má $E_L(1) = 0$, takže $\delta_L \approx 0$).
- $t = 2$: $\delta_L(2) = \delta_F(1) \cdot A_{FL} \cdot E_L(6) = 0, 5 \cdot 1, 0 \cdot 1, 0 = 0, 5$.

Viterbi nám opäť potvrdí cestu $F \rightarrow L$. Keďže sa cesta nezmenila, naše parametre zostanú rovnaké. **Algoritmus konvergoval.**

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM) s využitím Viterbiho tréovania

Prečo je to "Hard EM"? (Zhrnutie a varovanie)

Tento prístup je veľmi intuitívny, ale má jednu slabinu, o ktorej píše aj váš text:

1. **Je "tvrdý":** Povedali sme: "Táto cesta je jediná správna" (Viterbi cesta). Tým sme úplne ignorovali možnosť, že sme v čase $t = 2$ mohli byť aj v stave F s menšou pravdepodobnosťou. Baum-Welch algoritmus by vzal do úvahy **všetky** možné cesty (vážený priemer), čo je matematicky presnejšie.
2. **Rýchlosť:** Viterbiho tréovanie je veľmi rýchle (pretože len spočítame najlepšiu cestu), zatiaľ čo Baum-Welch vyžaduje zložitejšie sčítanie všetkých ciest cez Forward a Backward algoritmus.
3. **Výsledok:** Často skončí v **lokálnom maxime**. Ak si pri inicializácii vyberieme zlé parametre, algoritmus sa "zasekne" v riešení, ktoré nie je globálne najlepšie, pretože sa rozhodol iba pre jednu cestu a tú sa snaží vylepšovať.

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM): Baum-Welchov algoritmus

Baum-Welchov algoritmus (známy aj ako "Soft EM") je podstatne elegantnejší a štatisticky robustnejší ako Viterbiho tréning. Kým Viterbi sa rozhodne pre jednu "najlepšiu" cestu a všetko ostatné zahodí, Baum-Welch povie: **"Viem, že v čase $t = 1$ som bol v stave F na 62 % a v stave L na 38 %."**

Namiesto pridávania celých jednotiek k počtom (ako vo Viterbi), pridávame **zlomky pravdepodobností**.

1. Nastavenie (Inicializácia)

Použijeme rovnaký "naivný" model s pravdepodobnosťami 0,5 pre všetko, aby sme videli, ako sa algoritmus učí.

- **Pozorovaná sekvencia:** $x = \{1, 6\}$
- **Naše počiatočné parametre:** Všetky prechody (A) a všetky emisie (E) sú 0,5.

2. Iterácia: E-krok (Expectation)

V E-kroku musíme zistiť: "Aká je očakávaná hodnota, že sme boli v stave k a že sme prešli zo stavu k do l ?"

a) Dopredný a Spätný prechod (Forward-Backward):

Ako sme vypočítali v predchádzajúcich príkladoch (s uniformnými parametrami 0,5):

- $f_F(1) = 0,25$, $f_L(1) = 0,25$
- $f_F(2) = 0,125$, $f_L(2) = 0,125$
- $P(x) = 0,125 + 0,125 = 0,25$ (Celková pravdepodobnosť)
- Spätné hodnoty b nám vyjdú rovnako (symetria): $b_F(1) = 0,5$, $b_L(1) = 0,5$, $b_F(2) = 1$, $b_L(2) = 1$.

b) Výpočet posteriorných pravdepodobností (γ):

Toto je "srdce" algoritmu. $\gamma_k(t)$ je pravdepodobnosť, že v čase t sme v stave k .

$$\gamma_k(t) = \frac{f_k(t) \cdot b_k(t)}{P(x)}$$

- $\gamma_F(1) = \frac{0,25 \cdot 0,5}{0,25} = 0,5$
- $\gamma_L(1) = \frac{0,25 \cdot 0,5}{0,25} = 0,5$

(V prvej iterácii si model myslí, že je 50/50, či je kocka férová alebo falošná.)

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM): Baum-Welchov algoritmus

3. Iterácia: M-krok (Maximization)

Teraz aktualizujeme pravdepodobnosti. **Nepočítame kusy, počítame váhy.**

a) Nové emisné pravdepodobnosti (E):

Vzorec: $e_k(b) = \frac{\sum_{t: x_t=b} \gamma_k(t)}{\sum_t \gamma_k(t)}$

- Pre stav F , znak "1":
 - Čítateľ: $\gamma_F(1) = 0,5$
 - Menovateľ: $\gamma_F(1) + \gamma_F(2) = 0,5 + 0,5 = 1,0$
 - Nové $e_F(1) = 0,5/1,0 = \mathbf{0,5}$
- Pre stav L , znak "6":
 - Čítateľ: $\gamma_L(2) = 0,5$
 - Menovateľ: $\gamma_L(1) + \gamma_L(2) = 1,0$
 - Nové $e_L(6) = 0,5/1,0 = \mathbf{0,5}$

(Tu vidíme, že ak sú všetky stavy rovnako pravdepodobné, model sa "neučí". Ale ak by napr. $\gamma_L(2)$ bolo 0,8, nové $E_L(6)$ by stúplo na 0,8, čo by model "zistil", že falošná kocka hádže 6-ky.)

b) Nové prechodové pravdepodobnosti (A):

Vzorec pre prechod zo stavu k do l využíva očakávaný počet prechodov (ξ):

$$a_{kl} = \frac{\sum_t \xi_{kl}(t)}{\sum_t \gamma_k(t)}$$

Kde $\xi_{kl}(t)$ je pravdepodobnosť, že sme v čase t v stave k a v čase $t+1$ v stave l .

$$\xi_{kl}(t) = \frac{f_k(t) \cdot A_{kl} \cdot E_l(x_{t+1}) \cdot b_l(t+1)}{P(x)}$$

- Príklad pre A_{FL} :
 - $\xi_{FL}(1) = \frac{0,25 \cdot 0,5 \cdot 0,5 \cdot 1}{0,25} = 0,25$
 - Nové $a_{FL} = \frac{0,25}{0,5} = \mathbf{0,5}$

Prečo je Baum-Welch lepší?

1. **Žiadne zahadzovanie informácií:** Kým Viterbi "videl" len cestu $F \rightarrow L$, Baum-Welch videl štyri možnosti ($F \rightarrow F$, $F \rightarrow L$, $L \rightarrow F$, $L \rightarrow L$) a každej pridelil váhu podľa jej pravdepodobnosti.
2. **Jemné ladenie:** Vďaka tomu, že používame zlomky (váhy), model sa pri každej iterácii posúva o malý kúsok k lepšiemu riešeniu. Nevznikajú také drastické skoky ako vo Viterbi tréningu.
3. **Matematická garancia:** Baum-Welch (EM algoritmus) má matematickú záruku, že pri každej iterácii sa vierohodnosť $P(x|\theta)$ **zvyšuje alebo zostáva rovnaká**. Viterbiho tréning túto záruku nemá.

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM): Baum-Welchov algoritmus

Pre viac krokov (sekvencia $\{1, 6, 6\}$) sa Baum-Welch stáva veľmi zaujímavým. Zatiaľ čo pri dvoch krokoch sme len "ochutnali" algoritmus, pri troch krokoch už uvidíme, ako model pracuje so **závislosťami v čase**.

Keďže chceme, aby sa model skutočne niečo "naučil", použijeme tentoraz realistickejšie počiatkové odhady (tzv. "prior belief"). Predpokladáme, že stavy majú tendenciu zotrvať (kocka ostáva férová alebo falošná).

1. Počiatkové nastavenie (Inicializácia)

- **Pozorovanie (x):** $\{1, 6, 6\}$
- **Počiatkové pravdepodobnosti (π):** $\pi_F = 0,9, \pi_L = 0,1$
- **Počiatkové prechody (A):**
 - $A_{FF} = 0,7, A_{FL} = 0,3$
 - $A_{LF} = 0,3, A_{LL} = 0,7$
- **Počiatkové emisie (E):**
 - $e_F(1) = 0,5, e_F(6) = 0,5$
 - $e_L(1) = 0,2, e_L(6) = 0,8$ (Model "tuší", že falošná kocka hádže 6-ky radšej než 1-ty).

2. E-Krok (Expectation): Vypočítame "šance"

Tu využijeme Forward a Backward algoritmy, aby sme získali matice f (dopredné pravdepodobnosti) a b (spätné pravdepodobnosti).

Čas (t)	Pozorovanie	$f_F(t)$	$f_L(t)$
1	1	0,45	0,02
2	6	0,17	0,08
3	6	0,06	0,05

Poznámka: Hodnoty sú zaokrúhlené pre prehľadnosť. Celková pravdepodobnosť $P(x) \approx 0,11$.

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM): Baum-Welchov algoritmus

3. E-Krok: Výpočet posteriorných pravdepodobností

Teraz vytvoríme to "mäkké" (soft) rozhodovanie.

A) Pravdepodobnosť stavu v čase t ($\gamma_k(t)$):

$$\gamma_k(t) = \frac{f_k(t) \cdot b_k(t)}{P(x)}$$

- **Príklad pre $t = 2$:**

- $\gamma_F(2) = \frac{0,17 \cdot 0,35}{0,11} \approx 0,54$
- $\gamma_L(2) = \frac{0,08 \cdot 0,72}{0,11} \approx 0,46$
- *Interpretácia: V čase $t = 2$ model nie je istý, ale mierne sa prikláňa k Férovej kocke.*

B) Pravdepodobnosť prechodu ($t \rightarrow t + 1$): ($\xi_{kl}(t)$):

Toto je kľúčové pre Baum-Welch. Pozeráme sa na prechod z k do l medzi časmi t a $t + 1$.

$$\xi_{kl}(t) = \frac{f_k(t) \cdot A_{kl} \cdot E_l(x_{t+1}) \cdot b_l(t+1)}{P(x)}$$

4. M-Krok (Maximization): Aktualizácia parametrov

Teraz "prekreslíme" parametre modelu na základe našich vypočítaných váh (γ a ξ).

Nová emisná pravdepodobnosť $e_F(6)$:

Vzorec: $\frac{\sum \gamma_F(t) \text{ pre všetky časy, kde pozorovanie bolo 6}}{\sum \gamma_F(t) \text{ pre všetky časy}}$

- $t = 2$ a $t = 3$ sme pozorovali 6-ku.
- Počítame:

$$e_F(6)^{new} = \frac{\gamma_F(2) + \gamma_F(3)}{\gamma_F(1) + \gamma_F(2) + \gamma_F(3)} = \frac{0,54 + 0,55}{1,09 + 0,54 + 0,55} \approx 0,64$$

- **Čo sa stalo?** Pôvodne sme mali $e_F(6) = 0,5$. Algoritmus videl, že v čase $t = 2$ a $t = 3$ bola pomerne vysoká pravdepodobnosť, že sme v stave F , hoci padli 6-ky. Preto **zvýšil** odhad pravdepodobnosti, že Férová kocka hádže 6-ky.

Nová prechodová pravdepodobnosť A_{FF} :

Vzorec: $\frac{\sum \xi_{FF}(t)}{\sum \gamma_F(t)}$

- Ak nám vyšlo, že sme častejšie zotrúvali v stave F , A_{FF} sa zväčší (napr. z 0,7 na 0,75).

5. Posterior Decoding

Model mini kasíno

Expectation Maximization (EM): Baum-Welchov algoritmus

Zhrnutie tohto "Soft" prístupu

Prečo je to lepšie ako Viterbi?

1. **Žiadne tvrdé rozhodnutia:** Vo Viterbiho tréningu by sme museli povedať: "V čase 2 sme na 100 % v stave F". Baum-Welch povie: "V čase 2 sme na 54 % v stave F a na 46 % v stave L".
2. **Učenie zo všetkých ciest:** Tieto 46 % pravdepodobnosti stavu L prispeli k úprave parametrov pre falošnú kocku. Ak by sme použili Viterbi, tieto informácie by sa zahodili, pretože by sme ich "prekryli" silou stavu F .
3. **Konvergencia:** Po desiatkach takýchto iterácií sa pravdepodobnosti ustália na hodnotách, ktoré najlepšie vysvetľujú dáta.

Tento proces (Forward $\rightarrow$ Backward $\rightarrow \gamma/\xi$ výpočet $\rightarrow$ Update parametrov) sa opakuje dovtedy, kým sa "likelihood" (celková pravdepodobnosť modelu) prestane meniť.

5. Posterior Decoding

Aktuálne smery výskumu

- HMM sa vo veľkej miere používajú v rôznych oblastiach výpočtovej biológie. Jednou z prvých takýchto aplikácií bol algoritmus na vyhľadávanie génov známy ako GENSCAN, ktorý napísali Chris Burge a Samuel Karlin [1]. Pretože geometrické rozdelenie dĺžky HMM dobre nemodeluje exónové oblasti, Burge a kol. použili adaptáciu HMM známu ako skryté semi-Markovove modely (HSMM). Tieto typy modelov sa líšia v tom, že kedykoľvek je dosiahnutý skrytý stav, dĺžka trvania tohto stavu (d_i) je zvolená z rozdelenia a stav potom emituje presne d_i znakov. Prechod z tohto skrytého stavu do nasledujúceho je potom analógiou postupu HMM, okrem toho, že $a_{kk} = 0$ pre všetky k , čím sa zabráni prechodu do samého seba (samoprechodu). Mnohé z tých istých algoritmov, ktoré boli predtým vyvinuté pre HMM, možno modifikovať pre HSMM. Hoci detaily tu nebudeme rozoberať, dopredný a spätný algoritmus možno modifikovať tak, aby bežali v čase $O(K^2N^3)$, kde N je počet pozorovaných znakov. Táto časová zložitosť predpokladá, že neexistuje horná hranica dĺžky trvania stavu, ale stanovenie takejto hranice znižuje zložitosť na $O(K^2ND^2)$, kde D je maximálne možné trvanie stavu.
- Nedávno bolo vyvinuté úsilie, aby sa prístup založený na HMM pri vyhľadávaní homológií, nazvaný HMMER, stal životaschopnou alternatívou k BLASTu z hľadiska výpočtovej efektívnosti. Na rozdiel od väčšiny iných algoritmov na vyhľadávanie homológií, HMMER, ktorý napísal Sean Eddy, využíva spriemerovanie dopredného algoritmu (Forward algorithm) cez neistotu zarovnania, namiesto toho, aby uvádzal iba zarovnanie s maximálnou vierohodnosťou (ako vo Viterbiho algoritme); tento prístup je často lepší na detekciu vzdialenejších homológií, pretože s rastúcim časom divergencie môže existovať viac životaschopných spôsobov zarovnania sekvencií, z ktorých každý jednotlivý nie je dostatočne silný na to, aby bol odlíšený od šumu, ale spoločne poskytujú dôkaz o homológii. Mimoriadne vzrušujúcim nedávnym vývojom je, že HMMER je teraz dostupný ako webový server; možno ho nájsť na <http://www.ebi.ac.uk/Tools/hmmer/>.
- Zaujímavou témou, ktorú možno tiež preskúmať, je zhoda Viterbiho a posteriorných dekódovacích ciest; nielen pre detekciu CpG ostrovčekov, ale dokonca aj pre detekciu stavu chromatinu. Možno sa pozrieť na viacero ciest pomocou vzorkovania (samplingu) a klásť otázky ako:
 - Aký je rozdiel medzi cestou s maximálnou posteriornou pravdepodobnosťou (MAP) a Viterbiho cestou? Kde sa líšia?
 - Dajú sa nájsť úplné, ale maximálne disjunktné (od Viterbiho) cesty?